Phase 7 Summary: Documentation, Discussion & Write-Up

The Full Documentation Statement for Each Phase

Michael Manolakis

Spring 2026

Table of contents

1 Phase 1 Summary: Spatial Parsing & Economic Baseline Valuation **8**
1.1 Overview . . . 8
1.2 Script Inventory . . . 8
1.3 Pipeline Workflow . . . 9
1.3.1 Step 1 — Spatial Parsing & County Assignment . . . 9
1.3.2 Step 2 — Economic Proxy Merge . . . 9
1.3.3 Step 3 — RUCC Classification & Baseline Valuation . . . 9
1.4 Output Comparison & Discrepancy Analysis . . . 10
1.4.1 Baseline Valuation Statistics (non-missing courses) . . . 11
1.5 Key Changes Made (Ground-Up Revisions) . . . 11
1.6 File Inventory . . . 12
1.7 Next Steps . . . 12
1.8 Code Standardization Pass . . . 13
1.8.1 Conventions applied across all 14 scripts . . . 13
1.8.2 Removals . . . 13
1.8.3 Architectural change: Julia master . . . 14
1.8.4 RUCC data source split (unchanged, documented) . . . 14
1.9 CLAUDE.md Compliance Review . . . 14
1.9.1 Fixes Applied . . . 14
1.9.2 Observations (No Fix Required) . . . 15
1.10 Cross-Language Consistency Review . . . 16
1.10.1 What is Consistent . . . 16
1.10.2 Schema Discrepancies . . . 16

2 Phase 2 Summary: OSM Polygon Extraction & Acreage Matching **18**
2.1 Step 1 — OSM Golf Course Polygon Extraction . . . 18
2.1.1 What it does . . . 18
2.1.2 Key results . . . 18
2.1.3 Acreage summary (filtered polygons) . . . 18
2.2 Step 2 — Matching OSM Polygons to Phase 1 Courses . . . 19
2.2.1 What it does . . . 19
2.2.2 Key results (Cross-Language Parity) . . . 19
2.2.3 Matched acreage summary . . . 19
2.3 Tigris Landmarks Fallback Matching Attempt . . . 20
2.3.1 Overview . . . 20
2.3.2 Key Finding: API Change . . . 20
2.3.3 Future Considerations . . . 20
2.4 Phase 2 Refinement: Final Acreage Preparation for Imputation . . . 20
2.4.1 What it does . . . 21
2.4.2 Key results (Final Phase 2 Summary) . . . 21
2.4.3 Final data profile heading into Phase 3 . . . 21
2.5 File inventory . . . 21
2.6 CLAUDE.md Compliance Review . . . 22
2.6.1 `Phase_2.R` — Result . . . 22
2.6.2 `Phase_2.R` — Observations (No Fix Required) . . . 22
2.6.3 `Phase_2.jl` — Result . . . 23
2.6.4 `Phase_2.jl` — Fix Applied . . . 23
2.6.5 `Phase_2.jl` — Observation (No Fix Required) . . . 23
2.6.6 `Phase_2.py` — Result . . . 23
2.6.7 `Phase_2.py` — Fix Applied . . . 23
2.6.8 `Phase_2.py` — Observation (No Fix Required) . . . 24
2.7 Phase 2 Cross-Language Consistency Review . . . 24
2.7.1 What is Consistent . . . 24
2.7.2 Schema Discrepancies . . . 25
2.7.3 Key Observations . . . 25
2.8 Dependencies . . . 26
2.9 Next steps . . . 26
2.10 Code Standardization Pass . . . 26
2.10.1 Scripts standardized (11 total) . . . 27
2.10.2 Conventions applied across all 11 scripts . . . 27
2.10.3 Architectural change: Julia master . . . 28
2.10.4 Note on deprecated bulk scripts (Step 1 in Julia) . . . 28

3 Phase 3 Summary: MICE Imputation & Rubin's Rules Valuation **29**
3.1 Overview . . . 29
3.2 Script Inventory . . . 29
3.2.1 Master Scripts . . . 29

3.2.2 Bulk Test Sub-Scripts . . . 29
3.3 Step 1 — Multiple Imputation by Chained Equations (MICE) . . . 30
3.3.1 Missing Data Profile (Pre-Imputation) . . . 30
3.3.2 Imputation Method by Language . . . 30
3.3.3 Choice of M = 100 . . . 31
3.3.4 Predictor Variables Used in Imputation . . . 31
3.3.5 Why Random Forests / Tree-Based MICE? . . . 31
3.4 Step 2 — Aggregate Valuation & Rubin's Rules Pooling . . . 32
3.4.1 Per-Dataset Calculation . . . 32
3.4.2 Rubin's Rules Formulas Applied . . . 32
3.5 Step 3 — National Acreage Summary . . . 32
3.5.1 Purpose . . . 32
3.5.2 Pooling Formula (Acreage-Specific) . . . 33
3.5.3 Results — National Acreage (Pooled Across 100 Imputations) . . . 33
3.5.4 Cross-Language Column Note . . . 34
3.6 Results by Language . . . 34
3.6.1 Python — `Py_Rubins_Rules_Summary.csv` . . . 34
3.6.2 R — `R_Rubins_Rules_Summary.csv` . . . 35
3.6.3 Julia — `Jl_Rubins_Rules_Summary.csv` . . . 35
3.7 Cross-Language Comparison . . . 36
3.8 What the Data Tells Us . . . 36
3.8.1 1. The Aggregate Value is Robustly ~$940 Billion . . . 36
3.8.2 2. Between-Imputation Variance Dominates . . . 36
3.8.3 3. The 28.8% Acreage Imputation Did Not Destabilize Results . . . 36
3.9 Key Changes Made (Ground-Up Revisions) . . . 37
3.10 File Inventory . . . 37
3.11 Phase 3 Refinement: Complete Case Analysis (MICE-Free) . . . 38
3.11.1 Overview . . . 38
3.11.2 Results (MICE-Free Complete Case Analysis) . . . 38
3.11.3 Comparison with MICE Results . . . 38
3.12 Headline Result . . . 39
3.13 Code Standardization (April 30, 2026) . . . 39
3.13.1 Changes Applied Across All Scripts . . . 39
3.13.2 Files Updated . . . 40
3.14 National Acreage Summary Integration (May 1, 2026) . . . 41
3.14.1 New Scripts Created . . . 41
3.14.2 Bugs Fixed During Development . . . 42
3.14.3 Master Pipeline Integration . . . 42
3.14.4 Key Technical Addition: `pool_acreage()` Helper . . . 43
3.15 Phase 3A Structural Review — `Phase_3.R` . . . 43
3.15.1 What Passed . . . 43
3.15.2 Fixes Applied . . . 44
3.15.3 Observations (No Fix) . . . 45

3.16 Phase 3B Structural Review — `Phase_3.jl` . . . 45
3.16.1 What Passed . . . 45
3.16.2 Fixes Applied . . . 46
3.16.3 Observations (No Fix) . . . 47
3.17 Phase 3C Structural Review — `Phase_3.py` . . . 47
3.17.1 What Passed . . . 47
3.17.2 Fixes Applied . . . 48
3.17.3 Observations (No Fix) . . . 50
3.18 Phase 3D Cross-Language Consistency Review . . . 50
3.18.1 Consistency Results . . . 50
3.18.2 National Acreage Summary (Confirmed from Output CSVs) . . . 51
3.18.3 Observations . . . 52

4 Phase 4 Summary: Econometric Modeling **53**
4.1 Overview . . . 53
4.2 Model Specification . . . 53
4.3 Methodology . . . 54
4.3.1 Step 1 — Model Fitting . . . 54
4.3.2 Step 2 — Parameter Pooling (Rubin's Rules) . . . 54
4.4 Master Scripts . . . 54
4.4.1 `Phase_4.py` . . . 54
4.4.2 `Phase_4.R` . . . 55
4.4.3 `Phase_4.jl` . . . 55
4.5 Results . . . 55
4.5.1 Python — `Py_Regression_Results.csv` . . . 55
4.5.2 R — `R_Regression_Results.csv` . . . 56
4.5.3 Julia — `Jl_Regression_Results.csv` . . . 56
4.6 Cross-Language Coefficient Comparison *(M = 5 pilot — pending M = 100 rerun)* 57
4.7 What the Data Tells Us . . . 57
4.7.1 1. The Urban Land Premium is Large and Dominant . . . 57
4.7.2 2. More Holes = Higher Opportunity Cost . . . 57
4.7.3 3. Cross-Language Consistency Confirms the Pipeline . . . 57
4.7.4 4. The Fraction of Missing Information (FMI) Varies by Parameter . . . 58
4.7.5 5. R^2 is Moderate and Consistent . . . 58
4.8 Output File Summary . . . 58
4.9 Technical Notes . . . 59
4.9.1 HC1 Robust SE Implementations . . . 59
4.9.2 Serialization Formats . . . 59
4.9.3 Interactive Source vs. Command-Line Execution . . . 59
4.10 Bulk Test Scripts . . . 59
4.11 Historical / Legacy Scripts . . . 60
4.12 Code Standardization . . . 60
4.12.1 Four-Section Structure . . . 60

4.12.2 Variable Renaming . . . 60
4.12.3 Path Resolution . . . 61
4.12.4 [METHODOLOGY] Flags . . . 62
4.12.5 Language-Specific Conventions . . . 62
4.12.6 Folder Name Correction . . . 62
4.12.7 Outstanding Issues (Flagged, Not Fixed) . . . 62
4.13 Next Steps . . . 63
4.14 Phase 4C Script Review . . . 63
4.14.1 Compliance Audit . . . 63
4.14.2 Fixes Applied . . . 64
4.14.3 Observations (no fix) . . . 65
4.15 Phase 4D Cross-Language Consistency Review . . . 65
4.15.1 Full Parameter Name Divergence Table . . . 65
4.15.2 Coefficient Comparison (from actual M=100 Bulk Tests CSVs) . . . 66
4.15.3 Standard Error and FMI Comparison . . . 66
4.15.4 Checklist Items . . . 66
4.15.5 Critical Operational Observation . . . 67

5 Phase 5 Summary: Hawaii Micro-Case Study 68
5.1 Overview . . . 68
5.2 Methodology . . . 68
5.2.1 Data Sources . . . 68
5.2.2 Model Logic . . . 69
5.2.3 Phase 5b Pipeline Steps . . . 69
5.3 Results . . . 69
5.3.1 Phase 5a: Pilot Course Comparison (Manual Spot-Check) . . . 69
5.3.2 Phase 5b: Full Pipeline Results (Automated, Honolulu County) . . . 70
5.4 Key Findings . . . 72
5.5 Limitations . . . 73
5.6 Recommendations . . . 73
5.7 Implementation Notes . . . 74
5.7.1 Output File Conventions . . . 74
5.7.2 Scripts Inventory . . . 74
5.7.3 Dependencies . . . 75
5.8 Code Standardization . . . 76
5.8.1 Four-Section Structure . . . 76
5.8.2 Variable Renaming . . . 76
5.8.3 Path Resolution . . . 77
5.8.4 [METHODOLOGY] Flags . . . 77
5.8.5 Language-Specific Conventions . . . 78
5.8.6 Bugs Flagged (Not Fixed) . . . 78
5.8.7 Fixes Applied . . . 78

5.9 Julia Pipeline Implementation . . . 79
5.9.1 Data Flow (Standalone Master) . . . 79
5.9.2 Output Files (Standalone Master) . . . 80
5.9.3 Bugs Fixed in Julia Step Scripts . . . 80
5.9.4 Acreage Discrepancy Note . . . 82
5.10 Cross-Language Consistency Notes (Post-Audit) . . . 82
5.10.1 Oahu OSM Polygon Count . . . 82
5.10.2 Geographic Breakdown (Step 5) . . . 82
5.10.3 37-Course Ewa District Figure . . . 83

6 Phase 6 — Visualization **84**
6.1 Overview . . . 84
6.1.1 Language Assignment Strategy . . . 84
6.1.2 Last Verified Run — May 6, 2026 . . . 84
6.1.3 Structural Audit Log . . . 85
6.2 Master Scripts (Completed & Refactored) . . . 90
6.2.1 General Naming Scheme & Output Routing . . . 90
6.3 R Bulk Scripts . . . 91
6.3.1 Script 1 — `1_Macro_Maps.R` . . . 91
6.3.2 Script 2 — `2_County_Map.R` . . . 92
6.3.3 Script 3 — `3_Oahu_TMK_Map.R` . . . 92
6.3.4 Script 4 — `4_Oahu_Zoning_Map.R` . . . 93
6.3.5 Script 5 — `5_Econometric_Plots.R` . . . 93
6.3.6 Script 6 — `6_Advanced_Econometric_Plots.R` . . . 93
6.3.7 Script 7 — `7_Bivariate_Econometric_Map.R` . . . 94
6.3.8 Script 8 — `8_LaTeX_Tables.R` . . . 95
6.3.9 Script 9 — `9_Oahu_Opportunity_Cost_Map.R` . . . 95
6.3.10 Script 9b — `9b_Oahu_OC_Map_Rural_USDA_Sensitivity.R` . . . 96
6.3.11 Script 15 — `15_Residual_Map.R` . . . 97
6.4 R Output File Index . . . 98
6.5 Julia Bulk Scripts . . . 98
6.5.1 Script 5 — `5_Econometric_Plots.jl` . . . 98
6.5.2 Script 6 — `6_Advanced_Econometric_Plots.jl` . . . 99
6.5.3 Script 8 — `8_LaTeX_Tables.jl` . . . 99
6.5.4 Scripts 10–14 — Advanced Statistical Plots . . . 100
6.6 Output File Index . . . 100

7 Phase 7 Summary: Documentation, Discussion & Write-Up **103**
7.1 Thesis Coverage Summary — Phases 1 through 6 . . . 103
7.2 Phase 1 — Spatial Parsing & Economic Baseline Valuation . . . 103
7.2.1 Goal and Purpose . . . 103
7.2.2 Intent . . . 103
7.2.3 Accomplishments . . . 104

7.3 Phase 2 — OSM Polygon Extraction & True Acreage Matching 104
7.3.1 Goal and Purpose 104
7.3.2 Intent 104
7.3.3 Accomplishments 105
7.4 Phase 3 — MICE Imputation & Rubin's Rules Aggregate Valuation 105
7.4.1 Goal and Purpose 105
7.4.2 Intent 106
7.4.3 Accomplishments 106
7.5 Phase 4 — Econometric Modeling 107
7.5.1 Goal and Purpose 107
7.5.2 Intent 107
7.5.3 Accomplishments 108
7.6 Phase 5 — Hawaii Micro-Case Study 108
7.6.1 Goal and Purpose 108
7.6.2 Intent 109
7.6.3 Accomplishments 109
7.7 Phase 6 — Visualization 110
7.7.1 Goal and Purpose 110
7.7.2 Intent 110
7.7.3 Accomplishments 110
7.8 Cross-Phase Data Flow Summary 112
7.9 Audit and Quality Assurance Log 113

1 Phase 1 Summary: Spatial Parsing & Economic Baseline Valuation

1.1 Overview

Phase 1 establishes the baseline golf course dataset. The pipeline parses raw GPS coordinates, spatially assigns them to US Counties via point-in-polygon joins, and then merges economic proxy data (USDA Agricultural Land values and FHFA Residential Land values) using 2023 Rural-Urban Continuum Codes (RUCC) to produce a per-course `Baseline_Value_Per_Acre`.

To ensure robustness and cross-language replicability, the pipeline has been independently implemented in **three languages** — Python, R, and Julia — each producing its own prefixed output set.

1.2 Script Inventory

Script	Language	Outputs
`Phase_1.py`	Python	`Py_*` CSVs
`Phase_1.R`	R	`R_*` CSVs
`Phase_1.jl`	Julia	`Jl_*` CSVs

The Julia master script (`Phase_1.jl`) is an **orchestrator** — it `include()`s three modular sub-scripts from `Bulk Tests/Julia/`:

Sub-script	Function
`01_Parser.jl`	Parse raw CSV, extract `Course_Type` and `Holes` via regex, deduplicate
`02_SpatialJoin.jl`	Spatial point-in-polygon join to assign 5-digit FIPS county code
`03_Valuation.jl`	Merge USDA/FHFA proxies, apply RUCC classification, compute baseline

Python and R each implement all three steps inline within their master scripts, with `Bulk Tests/python/` and `Bulk Tests/R/` containing the corresponding modular component scripts.

1.3 Pipeline Workflow

1.3.1 Step 1 — Spatial Parsing & County Assignment

Loads `Golf Courses-USA.csv` from the read-only data backup. Extracts `Course_Type` and `Holes` via regex. Downloads official **2022 US Census County boundaries** via language-appropriate APIs (`pygris` for Python, `tigris` for R, `Shapefile.jl` for Julia) and performs a spatial point-in-polygon join to assign a `FIPS` code to every GPS coordinate.

- **Key fix applied:** A FIPS **zero-padding bug** was identified and resolved across all three language implementations. County FIPS codes stored as integers lost their leading zeros (e.g., `1001` instead of `"01001"`), causing join mismatches with the USDA and FHFA datasets. The fix enforces 5-digit zero-padded string formatting before any join operation.
- **Cross-language standardization:** The pipeline standardizes the spatial output by mapping 2-letter state abbreviations to a consistent `Tigris_State_Abbr` column across all languages, avoiding full-name / abbreviation inconsistencies.

1.3.2 Step 2 — Economic Proxy Merge

Cleans and standardizes two economic datasets, then joins both to the golf courses on 5-digit `FIPS`:

Dataset	Variable	Source
2022 USDA County Agricultural Land Values	`USDA_Ag_Value_Per_Acre`	USDA NASS Census of Agriculture
2024 FHFA Residential Land Prices	`FHFA_Res_Value_Per_Acre`	FHFA Expanded-Data HPI

1.3.3 Step 3 — RUCC Classification & Baseline Valuation

Fetches **2023 USDA Rural-Urban Continuum Codes (RUCC)** and classifies each course:

RUCC Code	Classification	Proxy Used
1 – 3	**Urban**	`FHFA_Res_Value_Per_Acre` (residential market)
4 – 9	**Rural**	`USDA_Ag_Value_Per_Acre` (agricultural market)

Courses that cannot be matched (missing county, data suppression, points outside CONUS boundaries) result in a missing `Baseline_Value_Per_Acre` — these are the target cases for MICE imputation in Phase 3.

1.4 Output Comparison & Discrepancy Analysis

All three pipelines were run on the same source data and compared for consistency. The FIPS zero-padding fix resolved the primary source of inter-language divergence.

Metric	Python (`Py_`)	R (`R_`)	Julia (`Jl_`)	Notes
Total Rows	16,297	16,292	16,292	Python retains 5 extra rows — different spatial join deduplication defaults in `geopandas` vs `sf`/`Shapefile.jl`
Missing FIPS	34	34	34	Consistent — points outside CONUS / over water
USDA Hit Rate	16,036	15,997	15,997	R & Julia match perfectly; Python captures a few dozen more due to row count
FHFA Hit Rate	11,722	11,719	11,717	All three match closely

Metric	Python (`Py_`)	R (`R_`)	Julia (`Jl_`)	Notes
Urban / Rural	11,391 / 4,872	11,386 / 4,872	11,386 / 4,872	R & Julia identical; Python +5 Urban from extra rows
Missing Baseline Value	**1,095**	**1,094**	**1,095**	**MICE Imputation target in Phase 3** — exceptionally consistent

1.4.1 Baseline Valuation Statistics (non-missing courses)

Statistic	Python	R	Julia
Min	$325.00	$325.00	$325.00
Median	$135,100.00	$133,700.00	$134,100.00
Mean	$413,699.57	$413,695.90	$413,700.97
Max	$24,324,800.00	$24,324,800.00	$24,324,800.00

The means converge to within $5 of each other across all three languages, confirming that the pipelines are statistically equivalent despite minor row-count differences at the margins.

1.5 Key Changes Made (Ground-Up Revisions)

1. **FIPS zero-padding bug fixed** in all three language implementations. Prior to the fix, FIPS integer coercion caused silent join mismatches that suppressed hundreds of USDA/FHFA assignments.
2. **Julia pipeline refactored into orchestrator + modules.** `Phase_1.jl` now calls the three modular `01_Parser.jl`, `02_SpatialJoin.jl`, `03_Valuation.jl` sub-scripts via `include()`, resolving world-age errors that occurred when defining and calling functions in the same top-level script scope in earlier versions.

3. **Statistical parity verified** across all three languages post-fix. Pre-fix, R and Julia diverged significantly from Python on USDA hit rates (~200+ row gap); post-fix, R and Julia align with each other to within 1–2 rows and Python diverges by only ~39 rows attributable to the `geopandas` spatial deduplication behavior.

1.6 File Inventory

File	Type	Description
`Phase_1.py`	Python Master	End-to-end pipeline in Python
`Phase_1.R`	R Master	End-to-end pipeline in R
`Phase_1.jl`	Julia Master	Orchestrator — calls `Bulk Tests/Julia/` sub-scripts
`Bulk Tests/Julia/01_Parser.jl`	Julia Module	Parsing & deduplication
`Bulk Tests/Julia/02_SpatialJoin.jl`	Julia Module	Point-in-polygon spatial join
`Bulk Tests/Julia/03_Valuation.jl`	Julia Module	Economic proxy merge & RUCC classification
`Bulk Tests/R/`	R Modules	Equivalent R modular sub-scripts
`Bulk Tests/python/`	Python Modules	Equivalent Python modular sub-scripts
`Py_Phase1_Baseline_Golf_Valuation.csv`	Output	Final Python baseline dataset
`R_Phase1_Baseline_Golf_Valuation.csv`	Output	Final R baseline dataset
`Jl_Phase1_Baseline_Golf_Valuation.csv`	Output	Final Julia baseline dataset
`{Py|R|Jl}_Phase1_Parsed_Golf_Courses.csv`	Intermediate	Post-parsing, pre-spatial-join
`{Py|R|Jl}_Phase1_Spatial_Joined_Golf_Courses.csv`	Intermediate	Post-spatial-join, pre-valuation

1.7 Next Steps

- **Phase 2:** Extract golf course polygon boundaries from the OpenStreetMap PBF file to calculate true physical acreage (`osm_acreage`).
- **Phase 3:** Run MICE imputation on the ~1,094–1,095 courses missing a baseline value, using spatial and structural covariates from Phases 1 and 2.

1.8 Code Standardization Pass

Scope: formatting and naming consistency only. No logic, formulas, spatial parameters, or filter thresholds were changed.

1.8.1 Conventions applied across all 14 scripts

Structure — mandatory four sections with two blank lines between them: `# === 1. LIBRARIES ===` / `# === 2. GLOBALS & PATHS ===` / `# === 3. FUNCTIONS ===` / `# === 4. EXECUTION ===`

Naming - Path constants → `ALL_CAPS` (`SCRIPT_DIR`, `ROOT_DIR`, `DATA_DIR`, `RAW_CSV`, `USDA_IN`, etc.) - Tabular frames → `_df` suffix (`courses_df`, `usda_df`, `fhfa_df`, `rucc_df`) - R spatial objects → `_sf` suffix (`courses_sf`, `county_sf`) - Python/Julia spatial objects → `_geo` suffix (`courses_geo`, `county_geo`) - Overwrite pattern preferred over throwaway intermediates (`parsed_data`/`cleaned_data`/`golf_spatial`/`golf_val`/`final_df` → all collapsed to `courses_df`)

Language-specific - R: `library(wooldridge)` + `library(tidyverse)` in every script; `readxl` kept explicitly; native pipe `|>` replacing `%>%`; `options(tigris_use_cache = TRUE)` in Section 2 - Python: `pathlib.Path(__file__).parent`-based paths; `_find_root()` removed everywhere - Julia: `@__DIR__` for `SCRIPT_DIR`; `main()` entry point with `if abspath(PROGRAM_FILE) == @__FILE__` guard

Methodology tags — `# [METHODOLOGY]` on spatial join operations in all three languages.

File existence checks — all local inputs checked before read; remote URLs receive no check.

1.8.2 Removals

Script	Removed
R bulk (all)	Redundant `library(dplyr/tidyr/stringr/readr)` sub-loads
R (all)	`%>%` pipe → `|>`
Python (all)	`_find_root()`, `import os`, `_HERE`/`ROOT`/`_data`/`_py` intermediates
Julia `01_Parser.jl`	`using Printf` (unused); dead `extract_course_type()` + `rest()` helper (produced `:Course_Type` which was immediately dropped in `select()`)

Script	Removed
Julia `02_SpatialJoin.jl`	`using ThreadsX` (never called; `Threads.@threads` is Base Julia)
Julia `03_Valuation.jl`	`using Downloads` (RUCC read from local CSV, not downloaded)
Julia bulk (all)	`run_parsing()` / `run_spatial_join()` / `run_valuation()` wrapper functions → replaced with `main()`

1.8.3 Architectural change: Julia master

The original `Phase_1.jl` orchestrated via `include()` + `run_parsing()`/`run_spatial_join()`/`run_valuation()`. Those wrapper functions were removed during bulk script standardization, breaking the master.

Resolution: `Phase_1.jl` rewritten as self-contained — all helper functions inlined, full pipeline in `main()` with per-step timing. Outputs write to `Data/Julia/` (not `Bulk Tests/Julia/`). Now matches the pattern of `Phase_1.R` and `Phase_1.py`.

1.8.4 RUCC data source split (unchanged, documented)

- R master / Python master: fetches live from USDA ERS URL
- Julia bulk + Julia master: reads from `00 - Data Sources/Secondary/2023-rural-urban-continuum-codes.csv` (local mirror; original USDA URL was dead, mirrored from WeitzGroup/SciMap-Methods on GitHub)

1.9 CLAUDE.md Compliance Review

Scope: structural and methodological compliance review of all three master scripts against CLAUDE.md standards. One fix applied per script; observations noted for awareness.

1.9.1 Fixes Applied

Script	Fix	Location
Phase_1.R	Added # [METHODOLOGY] CRS: EPSG 4326 (WGS 84)... above st_transform(4326)	Line 104
Phase_1.jl	Added # [METHODOLOGY] Spatial read - county boundaries in EPSG 4326... above GeoDataFrames.read(COUNTY_SHP)	Line 193
Phase_1.py	Added # [METHODOLOGY] CRS: EPSG 4326 (WGS 84)... above .to_crs("EPSG:4326")	Line 81

All three scripts now have # [METHODOLOGY] tags on every CRS transform and spatial file read, consistent across languages.

1.9.2 Observations (No Fix Required)

Script	Observation
Phase_1.R	future, furrr, parallelly loaded and plan(multisession) configured, but no furrr/future_map calls exist in Section 4. Unused parallel setup.
Phase_1.jl	ENV["JULIA_NUM_THREADS"] = "24" at line 19 is a runtime no-op; thread count must be set at launch (julia -t 24).
Phase_1.jl	Downloads.download(COUNTY_CB, COUNTY_ZIP) passes a local file path where a URL is expected — dead code path since COUNTY_SHP already exists.

Script	Observation
`Phase_1.py`	`extract_holes()` returns `18` as default when regex fails (line 53); R returns `NA` for the same case. Minor cross-language inconsistency on unparseable rows.
`Phase_1.py`	`as_is_col` hardcoded as `"Land Value\n(Per Acre, As-Is)"` inside `main()` (line 125); R uses `grep()` for dynamic column detection.

1.10 Cross-Language Consistency Review

Scope: Part 1D of the master review checklist. Compared all three baseline output CSVs for schema consistency, formula equivalence, CRS parity, and course count alignment.

1.10.1 What is Consistent

Check	Status
Raw input files	All three scripts read the same three source files and RUCC URL
Core economic fields	`FIPS`, `County_Name`, `Tigris_State_Abbr`, `USDA_Ag_Value_Per_Acre`, `FHFA_Res_Value_Per_Acre`, `RUCC_2023`, `county_type`, `Baseline_Value_Per_Acre` — identical column names in all three outputs
Baseline valuation formula	Urban → `FHFA_Res_Value_Per_Acre`; Rural → `USDA_Ag_Value_Per_Acre` — identical across all three
CRS	EPSG 4326 (WGS 84) in all three scripts
Course counts	R=16,292; Julia=16,292; Python=16,297 (the +5 Python rows are the documented `geopandas` deduplication difference)

1.10.2 Schema Discrepancies

Column	R	Julia	Python
`course_id`			missing

Column	R	Julia	Python
`Address`			missing
`City`			missing
`State_Abbr`			missing
`Zip_Code`			missing
`Details`	dropped	dropped	retained (raw unparsed column)
`Course_Name` content	Stripped (e.g., `"Seamountain Golf Course"`)	Raw with suffix (e.g., `"Seamountain Golf Course-HI"`)	Raw with suffix (same as Julia)

Highest-risk gap: `course_id` is absent from Python's output. If Phase 2 or Phase 3 Python scripts attempt to join or merge on `course_id`, they will fail with a KeyError. This must be verified in the Phase 2 Python review (Part 2C).

`Course_Name` divergence: R applies `str_remove(Name_State, "-.*$")` to strip the city/state suffix; Julia and Python carry the raw string. Any cross-language match on `Course_Name` would produce mismatches between R and Julia/Python. Downstream joins that use `FIPS` or `course_id` are unaffected.

2 Phase 2 Summary: OSM Polygon Extraction & Acreage Matching

2.1 Step 1 — OSM Golf Course Polygon Extraction

Scripts: `Phase_2.py`, `Phase_2.R`, `Phase_2.jl` **Outputs:** `Py_Phase2_OSM_Golf_Polygons.gpkg`, `R_Phase2_OSM_Golf_Polygons.gpkg`, `Jl_Phase2_OSM_Golf_Polygons.gpkg`

2.1.1 What it does

1. Extracts OSM areas tagged `leisure=golf_course`. (The Python script handles the direct pyosmium streaming of the 11 GB PBF to bypass GDAL corruption, while R and Julia read this verified extraction to maintain stability).
2. Converts WKB geometries into multipolygons wrapped in a GeoDataFrame (initial CRS: EPSG:4326).
3. **Reprojects** to EPSG:5070 (NAD83 / Conus Albers) so that acreage is calculated on a planar projection rather than meaningless square degrees.
4. Calculates true acreage using `area_m2 / 4046.8564224`.
5. Filters out mapping errors: drops polygons below 5 acres (fragments, individual holes) or above 1,500 acres (mega-resort blobs).
6. Saves the cleaned polygons as a GeoPackage (`.gpkg`), which natively handles the complex multipolygon geometries that CSVs cannot store.

2.1.2 Key results

Metric	Value
Processing time	~22.0 minutes (Python PBF Parse) / ~11 seconds (Julia/R)
Raw polygons captured	16,447
Geometry build errors	6
Dropped (< 5 or $> 1{,}500$ acres)	1,281
Final polygon count	**15,166**

2.1.3 Acreage summary (filtered polygons)

Statistic	Value
Min	5.0 acres
Median	127.8 acres
Mean	134.1 acres
Max	1,326.9 acres

2.2 Step 2 — Matching OSM Polygons to Phase 1 Courses

Scripts: `Phase_2.py`, `Phase_2.R`, `Phase_2.jl` **Outputs:** `Py_Phase2_Acreage_Matched.csv`, `R_Phase2_Acreage_Matched.csv`, `Jl_Phase2_Acreage_Matched.csv`

2.2.1 What it does

1. Loads the Phase 1 baseline dataset (`Phase1_Baseline_Golf_Valuation.csv`) and projects the point coordinates to EPSG:5070 to align with the polygons.
2. **Primary join** — `intersects`: finds courses whose point coordinate falls directly inside an OSM polygon.
3. **Fallback join** — `nearest` with `max_distance=500` meters: for any course that missed the primary join (e.g., the listed coordinate is a clubhouse or parking lot outside the mapped fairway), it grabs the nearest polygon's acreage if one exists within 500 m.
4. De-duplicates overlapping polygon matches.
5. Drops the heavy polygon geometries and saves a flat CSV with `osm_acreage` appended to all Phase 1 columns.

2.2.2 Key results (Cross-Language Parity)

Metric	Value
Phase 1 input rows	16,292
Direct intersect hits	5,458
Misses (need fallback)	10,834
Nearest-neighbor recoveries (within 500 m)	6,147
Total matched with osm_acreage	**11,605 (71.2%)**
Missing osm_acreage	**4,687 (28.8%)**

2.2.3 Matched acreage summary

Statistic	Value
Min	5.1 acres
Median	137.9 acres
Mean	147.6 acres
Max	1,326.9 acres

2.3 Tigris Landmarks Fallback Matching Attempt

Script: `Bulk Tests/R/02_Match_Tigris.R`

2.3.1 Overview

A fallback attempt was made to use the US Census Bureau's Tigris package (landmarks dataset) as an alternative source for golf course polygon acreage, in case OpenStreetMap coverage was incomplete. This approach aimed to identify "golf" or "country club" features and match them to courses missing OSM data.

2.3.2 Key Finding: API Change

The tigris R package underwent a significant API change between versions: - **Old behavior:** `landmarks()` could download all US landmarks without parameters - **New behavior (tigris 2.x):** The `landmarks(state = "XX")` function now requires a state parameter, requiring iterative downloads for each state

This architectural change significantly complicates the fallback matching approach and was not implemented in Phase 2.

2.3.3 Future Considerations

If Tigris landmarks are desired as a data source: 1. Modify `02_Match_Tigris.R` to loop through all states (or download national dataset via alternative means) 2. Filter for golf-related features using FULLNAME or FEATURE columns 3. Perform nearest-neighbor matching similar to Step 2

2.4 Phase 2 Refinement: Final Acreage Preparation for Imputation

Script: `Bulk Tests/R/03_Finalize_Acreage.R` **Output:** `R_Phase2_Acreage_Matched_v2.csv`

2.4.1 What it does

1. Loads the Step 1 OSM output file (`R_Acreage_Step1_OSM.csv`)
2. Assigns `"MICE_Target"` to remaining missing values in `acreage_source` column
3. Renames `OSM_Area_SqFt` to `final_acreage`
4. Saves the final pre-imputation dataset

2.4.2 Key results (Final Phase 2 Summary)

Acreage Source	Count	Percentage
MICE_Target	10,834	66.5%
OSM	5,458	33.5%
Total	**16,292**	**100%**

2.4.3 Final data profile heading into Phase 3

Variable	Missing count	% of 16,292
`Baseline_Value_Per_Acre`	1,095	6.7%
`final_acreage`	10,834	66.5%

The acreage gap is now clearly marked with `"MICE_Target"` in the `acreage_source` column to indicate which rows should be imputed during Phase 3 MICE (Multiple Imputation by Chained Equations).

2.5 File inventory

File	Type	Description
`Phase_2.py`	Python Script	Combined master script for Python pipeline
`Phase_2.R`	R Script	Combined master script for R pipeline (incorporates multi-processing)
`Phase_2.jl`	Julia Script	Combined master script for Julia pipeline (incorporates multi-threading)

File	Type	Description
`Bulk Tests/`	Directory	Modular source scripts (`01_ParseOSM`, `02_MatchOSM`, etc) for all languages
`*_Phase2_OSM_Golf_Polygons.gpkg`	Output	15,166 cleaned golf polygons (GeoPackages)
`*_Phase2_Acreage_Matched.csv`	Output	16,292 courses with `osm_acreage` appended (CSV files)

2.6 CLAUDE.md Compliance Review

Scope: structural and methodological compliance review of `Phase_2.R` and `Phase_2.jl` against CLAUDE.md standards.

2.6.1 `Phase_2.R` — Result

No violations found. No fixes applied.

2.6.2 `Phase_2.R` — Observations (No Fix Required)

Script	Observation
`Phase_2.R`	`PBF_FILE` in the script points to `00 - Data Sources/Original Data/us-260413.osm.pbf` but the Phase 2 summary header documents the PBF as residing in `Original Data - Backup/`. The script handles the mismatch via try/catch fallback to the Python GPKG, but the documentation path is misaligned.
`Phase_2.R`	`MAX_NEAREST_M <- 500` defines the nearest-neighbour cutoff but has no inline comment explaining the methodological basis for 500 m. Named constant is self-documenting; not a CLAUDE.md violation.
`Phase_2.R`	`rm(courses_sf, intersects_result, intersects_df, osm_golf_sf)` at line 337 (post Tier 1 cleanup) has no following `gc()` call. CLAUDE.md memory rule targets dataset-loading loops; this is post-join cleanup outside a loop, so not a strict violation, but several GB of spatial objects are freed without a GC hint.

2.6.3 `Phase_2.jl` — Result

One violation found. Fix applied.

2.6.4 `Phase_2.jl` — Fix Applied

Script	Fix	Location

`Phase_2.jl`	Added
	`courses_df.acreage_source`
	`=`
	`ifelse.(ismissing.(courses_df.osm_acreage),`
	`"MICE_Target",`
	`"OSM")`
	immediately after
	`courses_df.osm_acreage`
	`=`
	`acreage_results`
		After line 214 (original)

2.6.5 `Phase_2.jl` — Observation (No Fix Required)

Script	Observation
`Phase_2.jl`	No Tigris second tier: Phase_2.R runs a three-tier pipeline (OSM → Tigris landmarks → MICE_Target); Phase_2.jl is single-tier (OSM intersect + 500 m nearest → MICE_Target). Tigris cannot be replicated in Julia (`tigris` is an R-only package). The Julia MICE-target count will therefore be higher than R's. This is expected and not a violation.

2.6.6 `Phase_2.py` — Result

One violation found. Fix applied.

2.6.7 `Phase_2.py` — Fix Applied

Script	Fix	Location
Phase_2.py	Added `courses_geo["acreage_source"] = courses_geo["osm_acreage"].apply(lambda x: "MICE_Target" if pd.isna(x) else "OSM")` after the de-duplication step	After line 188 (original)

2.6.8 `Phase_2.py` — Observation (No Fix Required)

Script	Observation
Phase_2.py	No Tigris second tier (same as Julia — R-only package). Two-value schema ("OSM" | "MICE_Target") is expected.
Phase_2.py	Part 1D flag resolved: `Phase_2.py` performs only spatial joins (`sjoin`, `sjoin_nearest`) and never joins on `course_id`. The missing `course_id` in `Py_Phase1_Baseline_Golf_Valuation.csv` causes no failure in Phase 2.

2.7 Phase 2 Cross-Language Consistency Review

Scope: Part 2D of the master review checklist. Compared all three Phase 2 master scripts for `acreage_source` schema consistency, match distance thresholds, MICE-target count alignment, output CSV column schemas, and GPKG CRS.

2.7.1 What is Consistent

Check	Status
Polygon match distance threshold	`MAX_NEAREST_M = 500` m in all three languages
Output GPKG CRS	EPSG:5070 (NAD83 Conus Albers) in all three

Check	Status
`acreage_source` column present	All three output CSVs now carry `acreage_source` after 2B/2C fixes
Acreage units in output CSVs	All three report acreage in acres

2.7.2 Schema Discrepancies

Column	R	Julia	Python
Primary acreage column name	`final_acreage` (coalesced OSM+Tigris)	`osm_acreage`	`osm_acreage`
`tigris_acreage`	retained in CSV	absent	absent
`acreage_source` values	"OSM" | "Tigris" | "MICE_Target"	"OSM" | "MICE_Target"	"OSM" | "MICE_Target"
MICE_Target count	Lower (Tigris recovers additional courses)	Higher	Higher
`course_id`, `Address`, `City`, `State_Abbr`, `Zip_Code`	(inherited from Phase 1 R)	(inherited from Phase 1 Julia)	missing (Phase 1 gap carries forward)

2.7.3 Key Observations

1. **`acreage_source` value set is asymmetric by design**: R produces three categories ("OSM" | "Tigris" | "MICE_Target"); Julia and Python produce two ("OSM" | "MICE_Target"). The `tigris` package is R-only; the two-value schema in Julia and Python is expected and correct. Phase 3 scripts should filter on `acreage_source != "MICE_Target"` rather than on a specific positive value to remain safe across all three languages.
2. **`final_acreage` vs `osm_acreage` — primary acreage column name differs**: R's pipeline coalesces OSM and Tigris acreage into a single `final_acreage` column. Julia and

Python write `osm_acreage` as the primary acreage column (OSM-only). CLAUDE.md's language-prefixed file separation rule (each Phase 3 script reads only its own `R_`, `Jl_`, or `Py_` output) means this does not break the pipeline — but Phase 3 scripts must reference the correct column name per language.

3. **MICE-target count asymmetry (explainable)**: R's Tigris Tier 2 recovers a subset of courses that remain unmatched after OSM processing. Julia and Python have no equivalent fallback. Consequently, R's final MICE_Target row count is lower than Julia's and Python's. The direction of this asymmetry is deterministic; the magnitude depends on the Tigris runtime download.

2.8 Dependencies

- **Python**: `osmium` (pyosmium), `geopandas`, `pandas`, `shapely`
- **R**: `sf`, `dplyr`, `readr`, `future`, `furrr`
- **Julia**: `ArchGDAL`, `GeoDataFrames`, `DataFrames`, `CSV`, `ThreadsX`

2.9 Next steps

- **Phase 3:** MICE imputation to fill the 4,687 missing `osm_acreage` values and 1,095 missing `Baseline_Value_Per_Acre` values using spatial, structural, and economic covariates.

2.10 Code Standardization Pass

Scope: formatting and naming consistency only. No logic, formulas, spatial parameters, or filter thresholds were changed.

Script	Language

2.10.1 Scripts standardized (11 total)

Script	Language
`Bulk Tests/R/00 - Parse_osm_Golf_Polygons.R`	R
`Bulk Tests/R/01_Match_OSM.R`	R
`Bulk Tests/R/02_Match_Tigris.R`	R
`Bulk Tests/R/03_Finalize_Acreage.R`	R
`Bulk Tests/python/parse_osm_golf_polygons.py`	Python
`Bulk Tests/python/match_osm_to_courses.py`	Python
`Bulk Tests/Julia/01_ParseOSM.jl`	Julia
`Bulk Tests/Julia/02_MatchOSM.jl`	Julia
`Phase_2.R`	R master
`Phase_2.py`	Python master
`Phase_2.jl`	Julia master

Four deprecated R prototypes (`area.R`, `match_osm_to_courses.R`, `mock_execute.R`, `polygons.R`) were confirmed unused and removed from the workflow by the user.

2.10.2 Conventions applied across all 11 scripts

Structure — mandatory four sections with two blank lines between them: `# === 1. LIBRARIES ===` / `# === 2. GLOBALS & PATHS ===` / `# === 3. FUNCTIONS ===` / `# === 4. EXECUTION ===`

Naming - Path/config constants → `ALL_CAPS` (`SCRIPT_DIR`, `ROOT_DIR`, `PHASE1_CSV`, `OUT_CSV`, `TARGET_CRS`, `MAX_NEAREST_M`, `MIN_ACRES`, `MAX_ACRES`, `SQ_M_PER_ACRE`, `SQ_FT_PER_ACRE`, `SAFE_WORKERS`, `ALL_STATES`) - R spatial objects → `_sf` suffix: `osm_golf_sf`, `tigris_golf_sf`, `courses_sf`, `miss_sf` - Python/Julia spatial objects → `_geo` suffix: `osm_golf_geo`, `courses_geo`, `miss_geo` - Tabular frames → `_df` suffix: `courses_df`, `acreage_df`, `baseline_df` - Helper functions → `print_separator` (R); `format_number` / `format_decimal` (Julia)

Methodology tags — `# [METHODOLOGY]` on spatial reads, joins, CRS transforms, and geometric operations in all three languages.

Language-specific - R: `library(wooldridge)` + `library(tidyverse)` in every script; redundant sub-library loads (`dplyr`, `readr`, `stringr`) removed; native pipe `|>` throughout; `options(tigris_use_cache = TRUE)` in Section 2 for any script using `tigris` - Python:

pathlib.Path(__file__).parent-based paths; import os and os.path removed everywhere; os.path.normpath() calls removed; path.exists() replaces os.path.exists(); OUT_*.parent.mkdir(parents=True, exist_ok=True) added before saves - Julia: using Statistics replaces import Statistics: median, mean; using ThreadsX removed (unused — Threads.@threads is Base Julia); module-level const constants; wrapper functions (run_parse_osm(), run_match_osm()) replaced by main() entry points

2.10.3 Architectural change: Julia master

Phase_2.jl was originally an orchestrator using include() + run_parse_osm() / run_match_osm(). Since wrapper functions are removed during bulk script standardization, the master was rebuilt as self-contained — all helper functions and both pipeline steps inlined into main(), matching the pattern of Phase_1.jl.

Additionally, ENV["JULIA_NUM_THREADS"] = "24" was removed from the top of the file. This assignment has no effect at runtime — Julia thread count must be set before the interpreter starts via julia -t N or the JULIA_NUM_THREADS environment variable.

2.10.4 Note on deprecated bulk scripts (Step 1 in Julia)

Phase_2.jl previously had run_parse_osm() commented out (Step 1 intentionally skipped at runtime). In the self-contained master, Step 1 is now active and reads from Data/Python/Py_Phase2_OSM_Golf_Polygons.gpkg (the pyosmium output). The Julia master requires the Python pipeline to have been run first to produce this input file.

3 Phase 3 Summary: MICE Imputation & Rubin's Rules Valuation

3.1 Overview

Phase 3 finalizes the aggregate national valuation of U.S. golf course land by addressing missing data through **Multiple Imputation by Chained Equations (MICE)** and pooling the resulting estimates using **Rubin's Rules**.

Two variables require imputation: - `osm_acreage` — the true polygon-derived acreage for each course (28.8% missing) - `Baseline_Value_Per_Acre` — the economic proxy value (6.7% missing)

The pipeline is independently implemented in **Python**, **R**, and **Julia**, each producing M = 100 complete imputed datasets plus a Rubin's Rules summary CSV.

3.2 Script Inventory

3.2.1 Master Scripts

Script	Language	Outputs
`Phase_3.py`	Python master	`Py_Imputed_Dataset_{1..100}.csv`, `Py_National_Acreage_Summary.csv`
`Phase_3.R`	R master	`R_Imputed_Dataset_{1..100}.csv`, `R_Rubins_Rules_Summary.csv`, `R_National_Acreage_Summary.csv`
`Phase_3.jl`	Julia master	`Jl_Imputed_Dataset_{1..100}.csv`, `Jl_Rubins_Rules_Summary.csv`, `Jl_National_Acreage_Summary.csv`

3.2.2 Bulk Test Sub-Scripts

Script	Language	Purpose
`Bulk Tests/python/run_mice_imputation.py`	Python	MICE imputation step only
`Bulk Tests/python/rubins_rules_pooling.py`	Python	Rubin's Rules pooling step only

Script	Language	Purpose
Bulk Tests/python/National_Acreage_Summary.py	Python	National acreage footprint summary
Bulk Tests/R/MICE.R	R	MICE imputation step only
Bulk Tests/R/rubins_rules_pooling.R	R	Rubin's Rules pooling step only
Bulk Tests/R/Phase_3_National_Acreage_Summary.R	R	National acreage footprint summary
Bulk Tests/R/Phase_3_Analysis_Suite_v2.R	R	Full suite analysis
Bulk Tests/R/Phase_3_Granular_Calculations.R	R	Granular per-division calculations
Bulk Tests/R/Phase_3_Granular_Pooling.R	R	Granular Rubin's pooling
Bulk Tests/R/Phase_3_MICE_Free_Analysis.R	R	Complete case (MICE-free) baseline
Bulk Tests/R/Phase_3_Selection_Bias_Check.R	R	Selection bias diagnostics
Bulk Tests/Julia/MICE.jl	Julia	MICE imputation step only
Bulk Tests/Julia/Rubins_Pooling.jl	Julia	Rubin's Rules pooling step only
Bulk Tests/Julia/National_Acreage_Summary.jl	Julia	National acreage footprint summary

---

3.3 Step 1 — Multiple Imputation by Chained Equations (MICE)

3.3.1 Missing Data Profile (Pre-Imputation)

Variable	Missing	% of ~16,292
osm_acreage	4,687	28.8%
Baseline_Value_Per_Acre	1,095	6.7%
county_type	34	0.2%

3.3.2 Imputation Method by Language

Language	Package	Algorithm	Parallelization
Python	`miceforest` v6.0.5	LightGBM Gradient-Boosted Random Forest	Native `miceforest` multithreading
R	`mice`	Random Forest (`method = "rf"`)	`futuremice()` via `furrr/future`
Julia	`Mice.jl`	Mice algorithm (Random Forest default)	Native Julia parallel processing

All three use **M = 100** imputed datasets and **10 MICE iterations** per dataset.

3.3.3 Choice of M = 100

While early literature (Rubin, 1987) suggested 3 to 10 imputations were sufficient for point estimates, modern computational standards (Graham et al., 2007; von Hippel, 2020) demonstrate that higher numbers are required to stabilize standard errors and eliminate Monte Carlo error. Given the 28.8% missingness rate in the spatial acreage data, this study utilized $M = 100$ imputed datasets. This computationally intensive approach ensures asymptotic stability in the between-imputation variance (V_B) and provides highly robust confidence intervals when pooled via Rubin's Rules.

3.3.4 Predictor Variables Used in Imputation

Variable	Role
`Holes`	Structural proxy (course size → acreage)
`Course_Type` / `Ownership_Type`	Public / Private / Municipal
`county_type`	Urban / Rural (RUCC-derived from Phase 1)
`Longitude` & `Latitude`	Spatial geography

3.3.5 Why Random Forests / Tree-Based MICE?

1. **Non-linearity:** Land values and course sizes have complex, non-linear relationships with geography — tree models capture this without manual feature engineering.
2. **No negative predictions:** Unlike linear models, tree-based algorithms predict strictly within the observed range, preventing impossible negative acreages or land values.
3. **Handles mixed types:** Categorical predictors (`Course_Type`, `county_type`) are handled natively without requiring manual dummy encoding.

3.4 Step 2 — Aggregate Valuation & Rubin's Rules Pooling

3.4.1 Per-Dataset Calculation

For each of the 100 imputed datasets within each language:

```
Total_Opportunity_Cost_i = osm_acreage × Baseline_Value_Per_Acre
Q_i = sum(Total_Opportunity_Cost_i)
Var_i = var(Total_Opportunity_Cost_i)
```

3.4.2 Rubin's Rules Formulas Applied

Symbol	Formula	Description
Q	`mean(Q … Q_M)`	Pooled point estimate
V_W	`mean(Var … Var_M)`	Within-imputation variance
V_B	`var(Q … Q_M, ddof=1)`	Between-imputation variance
V_T	`V_W + V_B + V_B/M`	Total variance
SE	`sqrt(V_T)`	Pooled standard error
95% CI	`Q ± 1.96 × SE`	Confidence interval

3.5 Step 3 — National Acreage Summary

3.5.1 Purpose

Acreage is a **fixed spatial measurement** (derived from OSM polygons), not a modelled quantity — it does not vary across imputed datasets because MICE imputes acreage itself, not geography. The purpose of this step is therefore to:

1. Report the total physical U.S. golf course footprint pooled across the 100 imputed datasets (as a sanity check on imputation stability)
2. Break that footprint down by `county_type` (Urban / Rural)
3. Quantify any residual between-imputation variance as a measure of imputation uncertainty in the spatial measurements

3.5.2 Pooling Formula (Acreage-Specific)

Because within-imputation variance is zero for a fixed spatial attribute, only **between-imputation variance** enters the confidence interval:

Symbol	Formula	Description
Q	`mean(total_acres … total_acres_M)`	Pooled acreage estimate
V_B	`var(total_acres … total_acres_M, ddof=1)`	Between-imputation variance
SE	`sqrt(V_B + V_B/M)`	Standard error (no within-variance term)
95% CI	`Q ± 1.96 × SE`	Confidence interval

3.5.3 Results — National Acreage (Pooled Across 100 Imputations)

3.5.3.1 Julia — `Jl_National_Acreage_Summary.csv`

County Type	Pooled Acres	SD (between)
Urban	1,697,432	—
Rural	591,446	—
Missing county_type	4,267	—
National Total	**2,293,146**	~0

3.5.3.2 Python — `Py_National_Acreage_Summary.csv`

County Type	Pooled Acres
Urban	1,699,917
Rural	601,698
National Total	**2,305,904**

3.5.3.3 R — `R_National_Acreage_Summary.csv`

Uses `final_acreage` column (R pipeline uses RUCC-derived acreage rather than raw OSM; see cross-language column note below).

3.5.4 Cross-Language Column Note

Language	Acreage Column	Source
Python	`osm_acreage`	Raw OSM polygon area
Julia	`osm_acreage`	Raw OSM polygon area
R	`final_acreage`	RUCC-adjusted / Phase 2 finalised acreage

The ~12,000-acre difference between Julia (~2.293 M) and Python (~2.306 M) reflects different MICE draws for the 28.8% of courses with missing acreage — within expected between-imputation variation, not a methodological discrepancy.

3.6 Results by Language

3.6.1 Python — `Py_Rubins_Rules_Summary.csv`

Dataset	Aggregate Value
Dataset 1	$939.744 B
Dataset 2	$946.782 B
Dataset 3	$940.406 B
Dataset 4	$942.505 B
Dataset 5	$945.688 B

Metric	Value
Pooled Aggregate (Q̄)	**$943.025 B**
Within-Imputation Variance (V_W)	2.1215e+16
Between-Imputation Variance (V_B)	9.7765e+18
Total Variance (V_T)	1.1753e+19
Standard Error	**$3.428 B**
95% CI	**$936.306 B — $949.744 B**

3.6.2 R — `R_Rubins_Rules_Summary.csv`

Dataset	Aggregate Value
Dataset 1	$936.497 B
Dataset 2	$935.091 B
Dataset 3	$936.156 B
Dataset 4	$942.558 B
Dataset 5	$929.930 B

Metric	Value
Pooled Aggregate (Q)	**$936.046 B**
Within-Imputation Variance (V_W)	2.1111e+16
Between-Imputation Variance (V_B)	2.0232e+19
Total Variance (V_T)	2.4300e+19
Standard Error	**$4.929 B**
95% CI	**$926.385 B — $945.708 B**

3.6.3 Julia — `Jl_Rubins_Rules_Summary.csv`

Dataset	Aggregate Value
Dataset 1	$952.240 B
Dataset 2	$955.778 B
Dataset 3	$952.705 B
Dataset 4	$949.874 B
Dataset 5	$946.349 B

Metric	Value
Pooled Aggregate (Q)	**$951.389 B**
Within-Imputation Variance (V_W)	2.1339e+16
Between-Imputation Variance (V_B)	1.2354e+19
Total Variance (V_T)	1.4846e+19
Standard Error	**$3.853 B**
95% CI	**$943.838 B — $958.941 B**

3.7 Cross-Language Comparison

Language	Pooled $\bar{Q}$	SE	95% CI Lower	95% CI Upper
Python	$943.025 B	$3.428 B	$936.306 B	$949.744 B
R	$936.046 B	$4.929 B	$926.385 B	$945.708 B
Julia	$951.389 B	$3.853 B	$943.838 B	$958.941 B
Consensus range	—	—	**~$936 B**	**~$959 B**

All three 95% confidence intervals overlap substantially, confirming **cross-language statistical agreement**. The spread between the three point estimates (~$15 B across a ~$940 B base) is less than 1.6% and attributable to different Random Forest RNG seeds and internal implementations across the three MICE backends — not a methodological inconsistency.

3.8 What the Data Tells Us

3.8.1 1. The Aggregate Value is Robustly ~$940 Billion

Despite three completely independent computational stacks (Python LightGBM, R Random Forest, Julia Mice.jl), all three converge on a national golf course land opportunity cost in the **$926 B – $959 B range**, with overlapping confidence intervals. The central estimate across all three is approximately **$943 billion**.

3.8.2 2. Between-Imputation Variance Dominates

In all three languages, V_B » V_W by roughly 2–3 orders of magnitude. This means the uncertainty in the estimate is driven almost entirely by **natural variation across imputed datasets** (i.e., the true uncertainty about missing values), not by measurement noise within individual datasets. This is the expected and correct behavior for a well-specified MICE model.

3.8.3 3. The 28.8% Acreage Imputation Did Not Destabilize Results

Despite `osm_acreage` being missing for 28.8% of courses (4,687 records), the standard errors across all 100 datasets within each language are remarkably tight (within-language coefficient of variation $< 0.5\%$). The Random Forest imputation successfully leveraged `Holes`, `county_type`, and GPS coordinates to produce stable acreage estimates for courses missing polygon data.

3.9 Key Changes Made (Ground-Up Revisions)

1. **R pipeline promoted to a full master script.** `Phase_3.R` was written as a complete, self-contained pipeline combining MICE (`futuremice`) and Rubin's Rules pooling, writing outputs directly to the Phase 3 root directory with the `R_` prefix. Previously, R only existed as Bulk Test sub-scripts.
2. **Julia pipeline rewritten for Mice.jl compatibility.** `Phase_3.jl` was refactored to use `Mice.jl`'s `mice()` + `complete()` API correctly, resolving world-age errors from prior versions that defined and called functions in the same top-level script execution context.
3. **Cross-language statistical parity verified.** All three languages were confirmed to produce overlapping 95% confidence intervals centered near $940 B, validating the tri-language approach as a robustness check.
4. **Output naming standardized.** All outputs are strictly prefixed (`Py_`, `R_`, `Jl_`) to prevent filename collisions and clearly attribute each dataset to its generating pipeline.

3.10 File Inventory

File	Description
`Phase_3.py`	Python master pipeline
`Phase_3.R`	R master pipeline
`Phase_3.jl`	Julia master pipeline
`Py_Imputed_Dataset_{1..100}.csv`	Python-imputed complete datasets
`R_Imputed_Dataset_{1..100}.csv`	R-imputed complete datasets
`Jl_Imputed_Dataset_{1..100}.csv`	Julia-imputed complete datasets
`Py_Rubins_Rules_Summary.csv`	Python pooled opportunity cost + CI
`R_Rubins_Rules_Summary.csv`	R pooled opportunity cost + CI
`Jl_Rubins_Rules_Summary.csv`	Julia pooled opportunity cost + CI
`Py_National_Acreage_Summary.csv`	Python pooled acreage footprint by county type
`R_National_Acreage_Summary.csv`	R pooled acreage footprint by county type
`Jl_National_Acreage_Summary.csv`	Julia pooled acreage footprint by county type
`Bulk Tests/python/`	Python modular sub-scripts
`Bulk Tests/R/`	R modular sub-scripts
`Bulk Tests/Julia/`	Julia modular sub-scripts

3.11 Phase 3 Refinement: Complete Case Analysis (MICE-Free)

Script: `Bulk Tests/R/Phase_3_MICE_Free_Analysis.R`

3.11.1 Overview

This refinement performs a complete case analysis that ignores all imputed data, providing a baseline comparison to the MICE-based results. It uses only courses with non-missing values for both `final_acreage` and `Baseline_Value_Per_Acre`.

3.11.2 Results (MICE-Free Complete Case Analysis)

Metric	Value
Courses in complete case analysis	**5,115**
Courses removed (missing data)	11,177

Aggregate Value	Amount
MICE-Free National Value	**$943.025 B**
MICE-Free Urban-Only Value	**$868.011 B**

3.11.3 Comparison with MICE Results

The MICE-free national value ($943.025 B) is remarkably close to the pooled MICE estimates across all three languages (range: $926–$959 B). This suggests that:

1. The 28.8% of courses missing `osm_acreage` are not systematically different in value from those with data
2. The Random Forest imputation successfully captured the underlying relationships

   Consensus finding: Despite excluding 68.4% of the sample (5,115 vs. ~16,292), the complete-case estimate falls within the MICE confidence intervals, confirming result robustness.

3.12 Headline Result

> **The aggregate opportunity cost of U.S. golf course land is robustly estimated at approximately $940 Billion (range: $926 B – $959 B), cross-validated independently across Python, R, and Julia pipelines using Rubin's Rules.**

These 300 imputed datasets (100 per language) are passed directly to **Phase 4** as the inputs for OLS econometric modeling.

3.13 Code Standardization (April 30, 2026)

All Phase 3 scripts (masters and bulk tests) were audited and updated for cross-language consistency. No formulas, filter thresholds, spatial parameters, or statistical logic were changed — only formatting and naming.

3.13.1 Changes Applied Across All Scripts

Convention	Before	After
File-level header	`"""..."""` (Python/Julia) or ad-hoc	`# Purpose / Inputs / Outputs` comment block
Section headers	Mixed / absent	`# === 1. LIBRARIES ===` through `# === 4. EXECUTION ===`
Path resolution (R)	`sys.frame(1)$ofile`, `dir.exists()` hacks, hardcoded `c:/Users/…`	`this.path::this.dir()`
Path resolution (Python)	`os.path.dirname(os.path.abspath(__file__))`	`pathlib.Path(__file__).parent`
Path resolution (Julia)	bare `script_dir = @__DIR__`	`const SCRIPT_DIR = @__DIR__`
Constants (R)	`safe_workers`, `script_dir`, `output_file`	`SAFE_WORKERS`, `SCRIPT_DIR`, `OUT_CSV`
Constants (Julia)	missing `const` on all globals	`const` on every module-level binding
Main dataset name	`data`, `df`, `raw`	`acreage_df`
Imputation subset	`imputation_data`	`imp_df`

Convention	Before	After
MICE result object	`imputed_results`, `kernel`, `result`	`imputed_list`
Rubin's summary df	`summary_df`, `final_summary`, `summary`	`pooled_df`
Included/excluded groups	`included`, `excluded`	`included_df`, `excluded_df`
Urban filter df	`df_urban`	`urban_df`
Rubin's variables	`Q_bar`, `V_W`, `V_B`, `SE`	`q_bar`, `v_w`, `v_b`, `se`, `ci_lo`, `ci_hi`
Pipe operator (R)	`%>%`	`|>`
Library loading (R)	`library(dplyr)` + `library(readr)`	`library(tidyverse)` + `library(this.path)`
Julia entry guard	bare `run_*()` at module level	`main()` + `if abspath(PROGRAM_FILE) == @__FILE__`
Julia function docstrings	`"""..."""` blocks	removed (no docstrings in bulk scripts)
`[METHODOLOGY]` flags	absent	added on: `futuremice()`, `mice()`, `miceforest`, `set.seed(42)`, all Rubin's Rules pooling blocks
File existence checks	partial	added for all input files and imputed dataset loops
Output directory creation	absent	`dir.create(…, recursive=TRUE)` (R), `mkdir(parents=True)` (Python), `mkpath()` (Julia)

3.13.2 Files Updated

R Masters - `Phase_3.R` — four-section structure; `SAFE_WORKERS`, `IMPUTE_COLS`, `OUT_DIR`/`OUT_CSV` constants; `acreage_df`, `imp_df`, `imputed_list`, `pooled_df`; snake_case Rubin's vars; `[METHODOLOGY]` flags; removed redundant `library(readr)`; `library(wooldridge)` retained (pre-existing)

Python Masters - `Phase_3.py` — `pathlib`; four-section structure; `SCRIPT_DIR`, `OUT_DIR`

constants; acreage_df, imp_df, imputed_list; [METHODOLOGY] flags; mkdir before writes; fixed folder name (& True Acreage → and True Acreage)

Julia Masters - Phase_3.jl — # comment header; const on all globals; main() entry guard; four-section structure; acreage_df, imp_df, imputed_list, pooled_df; snake_case Rubin's vars; [METHODOLOGY] flags; mkpath before writes

R Bulk Tests - MICE.R — this.path, acreage_df, imp_df, imputed_list, [METHODOLOGY] - rubins_rules_pooling.R — this.path, pooled_df, snake_case Rubin's vars, [METHODOLOGY] - Phase_3_Analysis_Suite_v2.R — this.path, all canonical names, DIVISION_MAP/ALL_DIVISIONS promoted to globals, snake_case Rubin's vars, [METHODOLOGY] - Phase_3_Granular_Calculations.R — this.path, meta_df, imp_df, DIVISION_MAP/ALL_DIVISIONS/OUT_RDS as constants - Phase_3_Granular_Pooling.R — this.path, pooled_df, snake_case Rubin's vars in rubins_rules(), [METHODOLOGY] - Phase_3_MICE_Free_Analysis.R — fixed hardcoded absolute path; acreage_df, urban_df; |> pipes - Phase_3_Selection_Bias_Check.R — fixed hardcoded absolute path; acreage_df, included_df, excluded_df, OUT_CSV; calc_stats moved to Functions section

Python Bulk Tests - run_mice_imputation.py — pathlib; four-section structure; acreage_df, imp_df, imputed_list; file existence check; [METHODOLOGY]; fixed folder name - rubins_rules_pooling.py — pathlib; four-section structure; pooled_df; snake_case Rubin's vars; fixed wrong header path (was Phase 1 Parsing, now Phase 3); [METHODOLOGY]

Julia Bulk Tests - MICE.jl — # comment header; const on all globals; main() guard; acreage_df, imputed_list; [METHODOLOGY]; fixed folder name - Rubins_Pooling.jl — # comment header; const on all globals; main() guard; pooled_df; snake_case Rubin's vars; [METHODOLOGY]; file existence check per dataset

---

3.14 National Acreage Summary Integration (May 1, 2026)

3.14.1 New Scripts Created

Three standalone bulk test scripts computing the total U.S. golf course physical footprint were created and then integrated into all three master pipelines:

Bulk Test Script	Language	Status
Bulk Tests/R/Phase_3_National_Acreage_Summary.R	R	Created
Bulk Tests/python/National_Acreage_Summary.py	Python	Created + debugged

Bulk Test Script	Language	Status
`Bulk Tests/Julia/National_Acreage_Summary.jl`	Julia	Created + debugged

3.14.2 Bugs Fixed During Development

Three Julia-specific runtime errors were diagnosed and resolved:

Error	Cause	Fix
`ArgumentError: column name :final_acreage not found`	Julia imputed datasets use `osm_acreage`, not `final_acreage` (R uses `final_acreage`)	Replaced all `final_acreage` references with `osm_acreage`
`TypeError: non-boolean (Missing) used in boolean context`	`county_type` is `Union{Missing, String}`; `==` propagates `Missing` instead of `false`	Changed `r.county_type == "Urban"` to `isequal(r.county_type, "Urban")`
`MethodError: no method matching pool_acreage(::SubArray{…})`	`DataFrames.combine` with a do-block passes a `SubArray` view, not a concrete `Vector{Float64}`	Changed type annotation from `Vector{Float64}` to `AbstractVector{<:Real}`

One Python error was also resolved:

Error	Cause	Fix
`KeyError: 'final_acreage'`	Same column name divergence as Julia	Changed `df["final_acreage"]` → `df["osm_acreage"]`

3.14.3 Master Pipeline Integration

The acreage summary was incorporated into all three master scripts as a new pipeline step, running after Rubin's Rules pooling on the already-generated imputed datasets (no re-imputation required):

Master Script	New Step	New Output
`Phase_3.R`	Step 3	`Data/R/R_National_Acreage_Summary.csv`
`Phase_3.jl`	Step 3	`Data/Julia/Jl_National_Acreage_Summary`
`Phase_3.py`	Step 2	`Data/python/Py_National_Acreage_Summar`

Note: Python's master was Step 2 (not Step 3) as it did not previously include a Rubin's Rules opportunity cost pooling step inline.

3.14.4 Key Technical Addition: `pool_acreage()` Helper

A dedicated helper distinct from the opportunity cost Rubin's Rules pooling was added to all three masters. It uses between-imputation variance only (no within-variance term), appropriate for a spatially-fixed attribute:

```
v_b = var(x)
se  = sqrt(v_b + v_b / M)
CI  = q_bar ± 1.96 × se
```

3.15 Phase 3A Structural Review — `Phase_3.R`

Scope: Part 3A of the master review checklist. Full structural, I/O, MICE methodology, and memory compliance review of `Phase_3.R`.

3.15.1 What Passed

Check	Detail
Four-section layout	Headers at lines 19, 32, 56, 71; two blank lines at all boundaries
No `library()` outside Section 1	All 7 libraries inside `suppressPackageStartupMessages`
ALL_CAPS constants + `this.path`	`SCRIPT_DIR`, `INPUT_CSV`, `OUT_DIR`, `M`, `IMPUTE_COLS`, `SAFE_WORKERS`
I/O paths and output files	`R_Phase2_Acreage_Matched_v2.csv` input; 100 imputed CSVs + 2 summary CSVs to `Data/R/`
File existence checks	`INPUT_CSV` guarded; Rubin's Rules loop guarded per-file

Check	Detail
M = 100	`M <- 100` at line 45; `for (i in 1:M)`
Seed documented	`set.seed(42)  # [METHODOLOGY]` at line 53; `parallelseed = 42` in `futuremice()`
No predictor leakage	Predictors: Holes, Course_Type/Ownership_Type, county_type, Lon, Lat — no OC leakage
MICE targets	MICE fills only NAs in `final_acreage` and `Baseline_Value_Per_Acre`; observed values untouched
`Baseline_Value_Per_Acre` in outputs	In `IMPUTE_COLS`; present in all 100 imputed CSVs
Identical output schemas	All 100 produced by `complete(imputed_list, i)` on same `imp_df`
`futuremice()` tagged	`# [METHODOLOGY]` at lines 120–121
Rubin's Rules tagged	`# [METHODOLOGY]` at lines 168–169

3.15.2 Fixes Applied

#	Location	Issue	Fix
1	Step 2 loop (lines 162–166)	`df` read each iteration but never freed — CLAUDE.md memory violation	Added `rm(df); gc()` before closing `}` of Rubin's Rules loop
2	Step 3 loop (lines 238–244)	`df_ac` read each iteration but never freed — CLAUDE.md memory violation	Added `rm(df_ac); gc()` before closing `}` of National Acreage loop
3	Step 3 line 246	`pool_acreage(acreage_totals)` is the acreage-specific Rubin's pooling block — no `# [METHODOLOGY]` tag	Added two-line `# [METHODOLOGY]` comment above the call

3.15.3 Observations (No Fix)

1. **Imputed CSVs contain 7 columns only**: `imp_df` subsets `acreage_df` to `predictors + IMPUTE_COLS` (Holes, course_col, county_type, Longitude, Latitude, final_acreage, Baseline_Value_Per_Acre). Geographic identifiers (FIPS, course_id, Course_Name, acreage_source) are absent. Phase 4 OLS needs only the 7 columns present; downstream phases should not expect identifier columns from the imputed datasets.
2. **`imputed_list` not freed after Step 1**: The `mids` object from `futuremice()` persists in memory through Steps 2 and 3 without `rm(imputed_list, imp_df); gc()`. CLAUDE.md's memory rule targets dataset-loading loops; this is a single large object, not a loop accumulation. Best-practice gap, not a violation.

3.16 Phase 3B Structural Review — `Phase_3.jl`

Scope: Part 3B of the master review checklist. Full structural, I/O, MICE methodology, and memory compliance review of `Phase_3.jl`.

3.16.1 What Passed

Check	Detail
Four-section layout	Headers at lines 18, 23, 42, 281; `main()` at lines 283–302; entry guard at line 304
All logic in `main()`	`run_imputation()`, `run_pooling()`, `run_acreage_summary()` called from `main()` only
`@__DIR__` paths; ALL_CAPS constants	`const SCRIPT_DIR = @__DIR__`; `INPUT_CSV`, `OUT_DIR`, `OUT_CSV`, `OUT_ACREAGE_CSV`, `M`, `IMPUTE_COLS`, `PREDICTOR_COLS` all `const` ALL_CAPS
No `Plasma.jl`	No reference anywhere in the file
I/O paths and outputs	`Jl_Phase2_Acreage_Matched.csv` input; 100 imputed CSVs + 2 summary CSVs to `Data/Julia/`
`isfile` guards	Three separate `isfile() || error()` guards in `run_imputation()`, `run_pooling()`, `run_acreage_summary()`
M = 100	`const M = 100`; `m_datasets` threaded through all three functions

Check	Detail
Seed documented	`Random.seed!(42)  # [METHODOLOGY]` at line 72
No predictor leakage	`PREDICTOR_COLS = [:Holes, :Course_Type, :county_type, :Longitude, :Latitude]`
`Baseline_Value_Per_Acre` in outputs	In `IMPUTE_COLS`; written to all 100 imputed CSVs
Rubin's Rules tagged	`# [METHODOLOGY]` at lines 139–140 in `run_pooling()`

3.16.2 Fixes Applied

#	Location	Issue	Fix
1	Header line 5	`Jl_Imputed_Dataset_{1..30}.csv` — stale M count	Updated to `{1..100}`
2	Header line 9	`m = 30 imputations` — stale M count	Updated to `m = 100 imputations`
3	Lines 35–36	Dev comment "run M=5, increase to 30 … 100 as a goal mark" — describes completed testing progression	Removed; `const M = 100` stands without the obsolete annotation
4	`run_pooling()` loop	`df` read each iteration but never freed — CLAUDE.md memory violation	Added `df = nothing; GC.gc()` before `@printf` inside the loop
5	`run_acreage_summary()` loop	`df` read each iteration but never freed — CLAUDE.md memory violation	Added `df = nothing; GC.gc()` after `by_type_list[i] = type_sums` (before `urban_acres` extraction, which depends only on `type_sums`)

#	Location	Issue	Fix
6	`run_acreage_summary()` line ~236	`pool_acreage(national_totals)` is acreage-specific Rubin's pooling — no `# [METHODOLOGY]` tag	Add a two-line `# [METHODOLOGY]` comment above the call, matching Phase_3.R Fix 3 pattern

3.16.3 Observations (No Fix)

1. **Julia imputed datasets save full Phase 2 schema**: `run_imputation()` writes `out = copy(acreage_df)` with imputed values merged back, producing CSVs with all Phase 2 columns. R's `complete(imputed_list, i)` returns only the 7-column `imp_df` subset. Julia's 100 imputed datasets are wider; Phase 4 Julia must reference `osm_acreage` (not `final_acreage`) and should be robust to additional columns.
2. **`ds1` freed on function return**: `ds1 = CSV.read("Jl_Imputed_Dataset_1.csv")` read for post-imputation verification is not explicitly freed, but it is a local variable inside `run_imputation()` — it goes out of scope and is eligible for GC when the function returns. No violation (contrast with R's global-scope `imputed_list` observation in Phase 3A).

3.17 Phase 3C Structural Review — `Phase_3.py`

Scope: Part 3C of the master review checklist. Full structural, I/O, MICE methodology, and memory compliance review of `Phase_3.py`.

3.17.1 What Passed

Check	Detail
Four-section layout	Sections at lines 11, 24, 45, 286; two blank lines at all boundaries

Check	Detail
`__file__` paths; ALL_CAPS constants	`SCRIPT_DIR = pathlib.Path(__file__).parent`; all constants (`INPUT_CSV`, `OUT_DIR`, `OUT_RUBINS_CSV`, `OUT_ACREAGE_CSV`, `M`, `IMPUTE_COLS`, `PREDICTOR_COLS`, `N_CORES`) in ALL_CAPS
I/O paths and outputs	`Py_Phase2_Acreage_Matched.csv` input; 100 imputed CSVs + 2 summary CSVs to `Data/python/`
File existence checks	`if not path.exists(): raise FileNotFoundError(...)` in all three functions
M = 100	`M = 100` at line 37; loop uses `range(m_datasets)`
Seed documented	`random_state=42` documented in `# [METHODOLOGY]` comment at lines 82–83
No predictor leakage	`PREDICTOR_COLS = ["Holes", "Ownership_Type", "county_type", "Longitude", "Latitude"]`; no OC variables
`Baseline_Value_Per_Acre` in outputs	In `IMPUTE_COLS`; assigned back in save loop
`ImputationKernel` and `.mice()` tagged	`# [METHODOLOGY]` at lines 82 and 89
IMPUTE_COLS column name	`osm_acreage` (correct for Python; matches Julia; different from R's `final_acreage`)

3.17.2 Fixes Applied

#	Location	Issue	Fix
1	Section 1	`gc` module never imported — `gc.collect()` would raise `NameError`; CLAUDE.md requires it in all dataset loops	Added `import gc` alphabetically before `multiprocessing`
2	`run_imputation()` save loop	`completed` and `out` created each of 100 iterations but never freed — CLAUDE.md memory violation	Added `del completed, out; gc.collect()` after `out.to_csv(fname, index=False)`

#	Location	Issue	Fix
3	`run_pooling()` loading loop	`df` read each of 100 iterations but never freed — same class as Phase_3.R Fix 1 and Phase_3.jl Fix 3	Added `del df; gc.collect()` after `within_vars.append(var_i)`
4	`run_acreage_summary()` loading loop	`df` read each of 100 iterations but never freed — same class as Phase_3.R Fix 2 and Phase_3.jl Fix 4	Added `del df; gc.collect()` after `by_type_frames.append(type_su` (safe: `urban`/`rural` downstream use `type_sums`, not `df`)
5	`run_pooling()` line ~162	No `# [METHODOLOGY]` above `q_bar = aggregates.mean()` — Rubin's Rules block untagged; same class as Phase_3.R Fix 3 and Phase_3.jl Fix 5	Added two-line `# [METHODOLOGY]` comment above `q_bar`
6	`run_acreage_summary()` line ~242	No `# [METHODOLOGY]` above `pool_acreage(national_totals)` — acreage-specific Rubin's pooling block untagged	Added two-line `# [METHODOLOGY]` comment above call
7	`run_acreage_summary()` line ~209	Function opened with `print("\n=== STEP 2: NATIONAL ACREAGE SUMMARY ===")` — execution section already prints "STEP 3" before calling the function, creating a duplicate with wrong number	Removed stale print from function body; execution-section label is authoritative

3.17.3 Observations (No Fix)

1. **Python imputed datasets save full Phase 2 schema**: Like Julia (Part 3B Observation 1), `run_imputation()` writes `out = acreage_df.copy()` with imputed values merged back, producing CSVs with all Phase 2 columns. Not a violation; Phase 4 Python should reference `osm_acreage` (not `final_acreage`) and should be agnostic to additional columns.
2. **`PREDICTOR_COLS` uses `"Ownership_Type"` (Python) vs `"Course_Type"` (Julia/R)**: Python's predictor set references `Ownership_Type` directly (column as it appears in the Phase 2 output); Julia casts `acreage_df.Ownership_Type` to a categorical column named `Course_Type`; R detects either `"Course_Type"` or `"Ownership_Type"` dynamically. The underlying data is the same; the column name difference is handled within each language's preparation step. No downstream impact given language-prefixed file separation.
3. **`imputed_list` (`ImputationKernel`) not freed after save loop**: The `mf.ImputationKernel` object holding all 100 imputed datasets persists through `run_pooling()` and `run_acreage_summary()` if those functions are called inside the same Python process. However, all three functions are called from `__main__` sequentially, and `imputed_list` is a local variable inside `run_imputation()` — it goes out of scope when the function returns. Python's reference counting frees it then. No violation.

3.18 Phase 3D Cross-Language Consistency Review

Scope: Part 3D of the master review checklist. Cross-language consistency check comparing `Phase_3.R`, `Phase_3.jl`, and `Phase_3.py` for imputation targeting, predictor equivalence, M count, column presence, summary CSV structure, and actual national acreage totals from the three output CSVs.

No fixes applied. All items confirmed consistent (with documented asymmetries from Phase 2D) or flagged as observations.

3.18.1 Consistency Results

Item	Status	Notes
MICE targeting logic	Consistent	All three rely on NA pattern in acreage column (logically equivalent to `acreage_source == "MICE_Target"`); cross-language target count asymmetry is Phase 2D's documented Tigris finding
Predictor variable set	Equivalent	All five predictors present: {Holes, ownership type, county_type, Longitude, Latitude}; column name for ownership differs (R dynamic, Julia alias, Python direct) but underlying data identical
M = 100	Confirmed	`M <- 100` (R), `const M = 100` (Julia), `M = 100` (Python)
`Baseline_Value_Per_Acre` in all 300 datasets	Confirmed	In `IMPUTE_COLS` in all three; written to every imputed CSV
Rubins_Rules_Summary Metric strings	Identical	Hard-coded identically in all three (same 8+M row structure); **note**: item 5 of checklist conflates Phase 3 aggregate summaries with Phase 4 regression coefficients — "Intercept, Holes, Urban County" appear only in Phase 4 output CSVs
National acreage totals in plausible range	Confirmed	R = 2,303,152 / Julia = 2,291,064 / Python = 2,306,485 — 0.67% spread across three independent MICE backends

3.18.2 National Acreage Summary (Confirmed from Output CSVs)

Language	National Total	Urban	Rural	NA/empty county
R	2,303,152 acres	1,701,726	597,101	4,325 (labeled "NA")
Julia	2,291,064 acres	1,698,944	587,833	4,287 (blank label)
Python	2,306,485 acres	1,700,032	602,051	absent (pandas groupby drops NA)

Spread: 15,421 acres (0.67% of mean) — consistent with independent MICE variation across three different backends (Random Forest, LightGBM, Mice.jl).

Urban/Rural split consistent: Urban ~74%, Rural ~26% in all three languages.

3.18.3 Observations

1. **Julia scientific notation**: `Pooled_Acres` for the National Total row in `Jl_National_Acreage_Summary.cs` is written as `2.29106386e6` — Julia's default float formatting for large numbers. All standard CSV readers parse this correctly. Cosmetic difference only; data value is correct.
2. **Python NA-county groupby gap**: pandas `groupby("county_type")` drops NA keys by default (`dropna=True`). Python's Urban+Rural subtotals (2,302,083) do not sum to the national total (2,306,485); the 4,402-acre gap is NA-county courses counted in the national total but absent from the breakdown. R and Julia both include an explicit row for these courses. Python's national total is correct; only the by-type breakdown is incomplete. Phase 6 scripts reading `Py_National_Acreage_Summary.csv` should not assume the breakdown rows sum to the national total.
3. **Checklist item 5 framing**: The Part 3D checklist asks whether Phase 3's Rubins_Rules_Summary CSVs contain "Intercept, Holes, Urban County" — regression coefficient names. Those names are Phase 4 outputs. Phase 3's Rubins_Rules_Summary contains aggregate opportunity cost statistics. The relevant consistency check (identical Metric string names across all three scripts) is confirmed.

4 Phase 4 Summary: Econometric Modeling

4.1 Overview

Phase 4 implements the econometric modeling step of the golf course land valuation thesis. The pipeline fits an OLS regression with HC1 heteroskedasticity-robust standard errors on each of the M = 100 MICE-imputed datasets produced in Phase 3, then pools the estimates across imputations using **Rubin's Rules** to produce statistically valid inference under multiple imputation.

The phase is implemented across three languages (Python, R, Julia) and exists in two tiers:

Tier	Purpose	Scripts
Master Scripts (Phase 4 root)	Single-file, end-to-end pipeline	`Phase_4.py`, `Phase_4.R`, `Phase_4.jl`
Bulk Tests (sub-directories)	Modular two-step scripts for testing	`model_fitting.*` + `parameter_pooling.*`

4.2 Model Specification

```
Log_Opportunity_Cost ~ Holes + county_type
```

Variable	Type	Description
`Log_Opportunity_Cost`	Outcome	`log1p(osm_acreage × Baseline_Value_Per_Acre)`
`Holes`	Continuous	Number of holes at the golf course
`county_type`	Binary categorical	`Rural` (reference) vs. `Urban`

Robust SE: HC1 sandwich estimator, $(n / (n - k)) \times (X'X)^{-1} X' \mathrm{diag}(\hat{e}^2) X (X'X)^{-1}$

4.3 Methodology

4.3.1 Step 1 — Model Fitting

For each of the M = 100 imputed datasets: 1. Load the dataset from Phase 3 (`{Py|R|Jl}_Imputed_Dataset_{1..100}.csv`) 2. Compute derived variables: - `Total_Opportunity_Cost = osm_acreage × Baseline_Value_Per_Acre` - `Log_Opportunity_Cost = log1p(Total_Opportunity_Cost)` 3. Drop rows with any missing values in model columns (consistently 34 rows per dataset) 4. Fit OLS using language-native implementation 5. Compute HC1 robust standard errors 6. Serialize: coefficients, robust SEs, R^2, N, df_resid

4.3.2 Step 2 — Parameter Pooling (Rubin's Rules)

Symbol	Formula	Description
$\bar{Q}$	`mean(Q )`	Pooled point estimate
V_W	`mean(Var_i)`	Within-imputation variance
V_B	`var(Q , ddof=1)`	Between-imputation variance
V_T	`V_W + (1 + 1/M) × V_B`	Total variance
SE	`√V_T`	Pooled standard error
FMI	`(1 + 1/M) × V_B / V_T`	Fraction of Missing Information
df	Barnard & Rubin (1999)	Adjusted degrees of freedom

Significance: `*** p<.001  ** p<.01  * p<.05  . p<.1`

4.4 Master Scripts

4.4.1 `Phase_4.py`

- **Data source:** `Py_Imputed_Dataset_{1..100}.csv` (Python MICE via `miceforest`, LightGBM backend)
- **OLS engine:** `statsmodels.formula.api.ols` with `cov_type="HC1"`
- **Intermediate output:** `Py_model_results.pkl`
- **Final output:** `Py_Regression_Results.csv`

4.4.2 `Phase_4.R`

- **Data source:** `R_Imputed_Dataset_{1..100}.csv` (R MICE via `futuremice`, Random Forest)
- **OLS engine:** `lm()` with `sandwich::vcovHC(type="HC1")`
- **Path resolution:** `get_script_dir()` + `find_work_dir()` — works from interactive source and `Rscript` command line; uses `stop()` instead of `quit()` to avoid crashing the R interactive terminal on fatal errors
- **Intermediate output:** `R_model_results.rds`
- **Final output:** `R_Regression_Results.csv`

4.4.3 `Phase_4.jl`

- **Data source:** `Jl_Imputed_Dataset_{1..100}.csv` (Julia MICE via `Mice.jl`)
- **OLS engine:** `GLM.lm()` with manual HC1 sandwich estimator
- **HC1 implementation:** `(n/(n-k)) × (X'X)⁻¹ X'diag(ê²)X (X'X)⁻¹` — computed directly from `modelmatrix()` and `residuals()` for version stability
- **Path resolution:** `@__DIR__` + `find_work_dir()` walk
- **Intermediate output:** `Jl_model_results.jls`
- **Final output:** `Jl_Regression_Results.csv`

4.5 Results

> **Note:** The tables below reflect pilot runs at M = 5 imputations. They will be updated once the M = 100 pipelines complete across all three languages.

4.5.1 Python — `Py_Regression_Results.csv`

N per model: 16,258 | **Rows dropped:** 34 per dataset | *(M = 5 pilot)*

Parameter	Coef	SE	t	df_adj	p	Sig	FMI
Intercept	12.2822	0.0386	318.55	613.9	<.001	***	0.079
Holes	0.0474	0.0024	20.06	3133.8	<.001	***	0.032
C(county_type)[T.Urban]	4.1720	0.0226	184.80	25.9	<.001	***	0.392

Model R^2 across imputations: mean 0.77 (from original `Phase_4.py` run)

4.5.2 R — `R_Regression_Results.csv`

N per model: 16,258 | **Rows dropped:** 34 per dataset | *(M = 5 pilot)*

Parameter	Coef	SE	t	df_adj	p	Sig	FMI
(Intercept)	12.2292	0.0414	295.32	8,997.1	<.001	***	0.014
Holes	0.0525	0.0027	19.79	1,625.7	<.001	***	0.047
factor(county_type)Urban	4.0014	0.0251	159.71	34.8	<.001	***	0.339

Model R² across imputations: - Mean: 0.6988 | Min: 0.6942 | Max: 0.7030

4.5.3 Julia — `Jl_Regression_Results.csv`

N per model: 16,258 | **Rows dropped:** 34 per dataset | *(M = 5 pilot)*

Parameter	Coef	SE	t	df_adj	p	Sig	FMI
(Intercept)	12.2471	0.0388	315.30	2,112.1	<.001	***	0.040
Holes	0.0476	0.0024	19.92	6,641.5	<.001	***	0.019
county_type: Urban	4.1577	0.0201	206.61	426.8	<.001	***	0.095

Model R² across imputations: - Mean: 0.7304 | Min: 0.7280 | Max: 0.7339

Parameter	Python	R	Julia

4.6 Cross-Language Coefficient Comparison *(M = 5 pilot — pending M = 100 rerun)*

Parameter	Python	R	Julia
Intercept	12.282	12.229	12.247
Holes	0.0474	0.0525	0.0476
Urban Premium	4.172	4.001	4.158
Mean R^2	~0.770	0.699	0.730
N per model	16,258	16,258	16,258

4.7 What the Data Tells Us

4.7.1 1. The Urban Land Premium is Large and Dominant

Across all three languages the `county_type: Urban` coefficient is ~**4.0–4.2 log-units**, which on the log-scale means urban golf course land is worth approximately `exp(4.1)  60×` more than otherwise equivalent rural land. This is by far the largest effect in the model and is estimated with enormous precision ($t > 150$ in every implementation), reflecting the well-known urban land price gradient.

4.7.2 2. More Holes = Higher Opportunity Cost

The `Holes` coefficient is positive and highly significant (~0.047–0.053) across all languages. Each additional hole is associated with roughly a **4.7–5.3% increase** in log opportunity cost, consistent with larger courses commanding premium land in higher-value areas (course size and land value co-vary positively).

4.7.3 3. Cross-Language Consistency Confirms the Pipeline

The intercept (~12.23–12.28), Holes effect (~0.047–0.053), and Urban premium (~4.0–4.2) are consistent across all three independent implementations. The minor numerical differences arise from each language using a different MICE backend (Python `IterativeImputer`, R `mice`, Julia `Mice.jl`), which produce statistically equivalent but not numerically identical imputed datasets.

4.7.4 4. The Fraction of Missing Information (FMI) Varies by Parameter

The FMI quantifies how much imputation uncertainty contributed to each pooled standard error:

Parameter	Py FMI	R FMI	Jl FMI	Interpretation
Intercept	0.079	0.014	0.040	Low — intercept well-identified
Holes	0.032	0.047	0.019	Low — Holes rarely missing
Urban	0.392	0.339	0.095	Moderate — some imputation variance

The elevated Urban FMI in Python and R indicates that the `county_type` Urban dummy carries more between-imputation variability — likely because `county_type` is used as a predictor in MICE, and variation in its imputed values (for courses that lacked it) propagates into the Urban coefficient across imputations.

4.7.5 5. R^2 is Moderate and Consistent

The model explains **69–73% of the variance** in log opportunity cost across all imputed datasets and languages. This is strong for a two-predictor model on cross-sectional observational data, driven primarily by the Urban/Rural distinction. The remaining variance reflects course-level heterogeneity not captured by holes count alone (course quality, amenities, local demand, etc.).

4.8 Output File Summary

Language	Master Script	Model Object	Pooled CSV
Python	`Phase_4.py`	`Py_model_results.pkl`	`Py_Regression_Results.csv`
R	`Phase_4.R`	`R_model_results.rds`	`R_Regression_Results.csv`
Julia	`Phase_4.jl`	`Jl_model_results.jls`	`Jl_Regression_Results.csv`

All files are saved to the `Phase 4 Econometric Modeling/` directory (same folder as the master scripts). Bulk test sub-scripts in `Bulk Tests/{python,R,Julia}/` produce identically-named files in their respective subdirectories.

4.9 Technical Notes

4.9.1 HC1 Robust SE Implementations

Language	Method
Python	`statsmodels cov_type="HC1"`
R	`sandwich::vcovHC(model, type="HC1")`
Julia	Manual: `(n/(n-k)) × (X'X)` $^{-1}$ `X'diag(ê²)X (X'X)` $^{-1}$ via `modelmatrix()`

4.9.2 Serialization Formats

Language	Format	Extension
Python	`pickle` binary	`.pkl`
R	R native serialization	`.rds`
Julia	`Serialization.serialize`	`.jls`

4.9.3 Interactive Source vs. Command-Line Execution

The R and Julia master scripts include robust path-detection logic: - **R:** `get_script_dir()` checks `--file=` CLI argument first, then inspects `sys.frames()` for the `ofile` attribute set by `source()`. Falls back to `getwd()`. Fatal errors use `stop()` (not `quit()`) so the R interactive terminal is not terminated. - **Julia:** `@__DIR__` macro resolves the script directory automatically at parse time — works identically under `julia script.jl` and `include()`.

4.10 Bulk Test Scripts

The `Bulk Tests/` sub-directory contains modular two-step versions of the pipeline split across `model_fitting.*` and `parameter_pooling.*`. These exist for iterative development and cross-language comparison. They are functionally identical to the master scripts but write their outputs to their respective language sub-directory.

Language dir	Intermediate	Final CSV
`Bulk Tests/python/`	`Py_model_results.pkl`	`Py_Regression_Results.csv`
`Bulk Tests/R/`	`R_model_results.rds`	`R_Regression_Results.csv`
`Bulk Tests/Julia/`	`Jl_model_results.jls`	`Jl_Regression_Results.csv`

4.11 Historical / Legacy Scripts

Earlier exploratory scripts (`debug_compare`, `debug_dtypes`, `debug_params`, `Pooled_Log_Regression`) used a fuller 5-parameter model specification (`Holes + Course_Type + county_type`) and are preserved in the root of the `Phase 4 Econometric Modeling/` directory. They are **not part of the current standardized pipeline** but document the iterative porting process from Python to R. Results from those scripts (R^2 0.77, Urban ~4.09) reflect the richer specification and are consistent with the current simplified model.

4.12 Code Standardization

All 9 Phase 4 scripts were standardized for naming consistency, path resolution, and structural conventions. No logic, formulas, spatial parameters, or filter thresholds were changed.

4.12.1 Four-Section Structure

Every script follows the same section order:

```
# === 1. LIBRARIES ===
# === 2. GLOBALS & PATHS ===
# === 3. FUNCTIONS ===
# === 4. EXECUTION ===
```

R and Julia use `# (none)` in Section 3 where no helper functions are defined.

4.12.2 Variable Renaming

Old name	New name	Applies to
`results_list`	`model_results`	all scripts
`results_df`	`pooled_df`	all scripts
`df` (per-loop dataset)	`acreage_df`	all scripts
`se_vals`	`se`	R and Julia scripts
`Q_bar`, `V_W`, `V_B`, `V_T`, `SE`	`q_bar`, `v_w`, `v_b`, `v_t`, `se`	Python scripts
`script_dir`, `work_dir`, `phase3_dir`, `output_dir`	`SCRIPT_DIR`, `PHASE3_DIR`, `OUT_DIR`	all
`formula_str`	`FORMULA_STR`	R / Julia
`rds_path` / `output_csv`	`RDS_PATH` / `OUT_CSV`	R / Julia bulk
`model_rds_path` / `regression_csv`	`MODEL_RDS` / `OUT_CSV`	Phase_4.R master
`MODEL_PKL_PATH` / `REGRESSION_CSV`	`PKL_PATH` / `OUT_CSV`	Phase_4.py master
`find_work_dir()`	removed	all — replaced by relative path construction
`get_script_dir()`	removed	R bulk — replaced by `this.path`

4.12.3 Path Resolution

Fragile `find_work_dir()` / `get_script_dir()` traversal functions were removed from every script and replaced with single-line relative constructions anchored to the script's own location:

Language	Anchor	Example
R	`this.path::this.dir()`	`SCRIPT_DIR <- this.path::this.dir()`
Python	`pathlib.Path(__file__).parent`	`SCRIPT_DIR = pathlib.Path(__file__).parent`
Julia	`@__DIR__`	`const SCRIPT_DIR = @__DIR__`

Bulk test scripts (3 levels up to `2 - Work`): - R / Julia: `joinpath(SCRIPT_DIR, "..", "..", "..", "Phase 3 ...")` - Python: `SCRIPT_DIR.parents[2] / "Phase 3 ..."`

Master scripts (1 level up to `2 - Work`, then into `Data/<Language>/`): - R: `file.path(SCRIPT_DIR, "..", "Phase 3 ...", "Data", "R")` - Python: `SCRIPT_DIR.parent / "Phase 3 ..." / "Data" / "Python"` - Julia: `joinpath(SCRIPT_DIR, "..", "Phase 3 ...", "Data", "Julia")`

4.12.4 [METHODOLOGY] Flags

Three blocks carry [METHODOLOGY] flags in every script:

Block	Tag content
lm() / smf.ols() call	OLS - log-linear model for opportunity cost
HC1 vcov block	HC1 robust standard errors - heteroskedasticity-consistent; HC1 = n/(n-k) finite-sample correction
Rubin's Rules block	Rubin's Rules - Barnard & Rubin (1999) df approximation

4.12.5 Language-Specific Conventions

- **R:** stars() moved to Section 3 with Roxygen2 #' docstring. Pre-existing library(wooldridge) and library(broom) in Phase_4.R retained with # pre-existing dependency - do not remove comments.
- **Python:** stars() moved to Section 3 with """docstring""". All scripts wrapped with def main(): ... if __name__ == "__main__": main() guard.
- **Julia:** get_stars() moved to Section 3 with """docstring""". All scripts wrapped with function main() ... end + if abspath(PROGRAM_FILE) == @__FILE__ / main() / end guard. All module-level variables declared const.

4.12.6 Folder Name Correction

Bulk test scripts previously referenced "Phase 3 Economic Merge & MICE Imputation" (ampersand). Corrected to "Phase 3 Economic Merge and MICE Imputation" to match the actual folder name and the Phase 3 standardization convention.

4.12.7 Outstanding Issues (Flagged, Not Fixed)

1. **Path mismatch — bulk vs. master:** Bulk test scripts resolve imputed datasets from the Phase 3 root directory (Phase 3 .../). Master scripts resolve from Phase 3 .../Data/<Language>/. Pre-existing inconsistency — not fixed.
2. **final_acreage vs. osm_acreage:** Phase_4.R uses final_acreage as the acreage column (intentional R-side design choice); all other scripts use osm_acreage. Cross-language column inconsistency — not fixed.
3. **Pkg.add() in Julia scripts:** All Julia scripts call Pkg.add() on every execution. Should be removed once packages are installed, but left as-is pending confirmation.

4. **`library(broom)` in `Phase_4.R`:** Loaded but no explicit `broom` call is visible — may be unused. Retained as pre-existing dependency.

4.13 Next Steps

1. **Phase 5:** Hawaii Micro-Case Study — validate the model coefficients against municipal tax assessment data for Hawaii golf courses.
2. Consider extending the bulk framework to include the full 5-parameter specification (`Holes + Course_Type + county_type`) for cross-language parity with legacy results.

4.14 Phase 4C Script Review

Script reviewed: `Phase_4.py` (master script, 247 lines pre-fix, 250 lines post-fix)

4.14.1 Compliance Audit

#	Check	Result	Notes
1	Four-section layout	Pass	`# === 1-4. ===` at lines 8/18/38/49; two blank lines at all boundaries
2	ALL_CAPS constants in Section 2	Pass	`SCRIPT_DIR`, `PHASE3_DIR`, `OUT_DIR`, `PKL_PATH`, `OUT_CSV`, `FORMULA_STR`, `M`, `IMPUTED_PATHS`
3	Relative path via `Path(__file__).parent`	Pass	`SCRIPT_DIR = pathlib.Path(__file__).parent` at line 20
4	`if __name__ == "__main__"` guard	Pass	Lines 245–246

#	Check	Result	Notes
5	File existence check before loop	Pass	List comprehension checks all 100 before loop; `raise SystemExit(1)` if any missing
6	`import gc` in Section 1	**FIXED**	Missing — would have caused `NameError` on any `gc.collect()` call
7	`del df; gc.collect()` in model-fitting loop	**FIXED**	`acreage_df`, `model`, `result` never freed; added `del acreage_df, model, result; gc.collect()`
8	Dependent variable is log(OC), not log(acreage)	Pass	`np.log1p(Total_Opportunity_Cost)`, `FORMULA_STR` contains `Log_Opportunity_Cost`
9	`# [METHODOLOGY]` on OLS, HC1, Rubin's blocks	Pass	Inline at line 101 (OLS), line 102 (HC1), line 164 (Rubin's)
10	Rubin's Rules formula correct	Pass	`v_b = coef_df.var(ddof=1)`, `v_w = var_df.mean()`, `v_t = v_w + (1+1/m_i)*v_b`; Barnard & Rubin (1999) df; `2 * stats.t.sf(|t|, df_adj)`
11	FMI stored in output CSV	Pass	`FMI = lambda_` in `pooled_df`
12	No synthetic data	Pass	All values from real CSVs

4.14.2 Fixes Applied

Fix 1 — Added `import gc` to Section 1: The `gc` module was entirely absent from imports. CLAUDE.md mandates `gc.collect()` inside all dataset-loading loops; without `import gc` any such call would raise `NameError` at runtime. Added `import gc` alphabetically first in Section 1. Same class as Phase_3.py Fix 1.

Fix 2 — Added `del acreage_df, model, result; gc.collect()` at end of model-fitting loop body: The 100-iteration loop created three large objects per iteration (`acreage_df` = full CSV DataFrame, `model` = statsmodels OLS object, `result` = fitted model with HC1 SEs) but never freed them before the next iteration. All needed values were already captured in `model_data` (lightweight dict of pandas Series + scalars). Added cleanup after the print statement inside the loop. Same CLAUDE.md memory violation class as Phase_4.R Fix 1, Phase_4.jl Fix 3, Phase_3.py Fixes 2–4.

4.14.3 Observations (no fix)

1. **`"Python" vs "python" in path strings`**: `PHASE3_DIR` uses `"Data" / "Python"` (capital P) while Phase_3.py writes to `"Data" / "python"` (lowercase). Windows case-insensitive filesystem makes this harmless but inconsistent. Not a CLAUDE.md violation.
2. **`stars() lacks NaN guard`**: R's `stars()` checks `is.na(x)` first; Julia's `get_stars()` checks `isnan(p)` first; Python's goes directly to `< 0.001`. In Python, `NaN < 0.001` evaluates to `False` (IEEE 754), so the function returns `""` for NaN inputs correctly. Not a functional bug — no fix applied.
3. **`C(county_type)[T.Urban] parameter name`**: statsmodels' `C()` operator produces this name in `pooled_df`. Diverges from R (`factor(county_type)Urban`) and Julia (`county_type: Urban`). Flagged for Part 4D.
4. **`log1p vs log`**: Dependent variable uses `np.log1p(Total_Opportunity_Cost)` rather than `np.log()`. Guards against zero values. Consistent with R and Julia. Phase 6 axis labels should technically read `log(1 + Opportunity_Cost)`. Flagged for Phase 6 label review.

4.15 Phase 4D Cross-Language Consistency Review

Source files read: `Bulk Tests/R/R_Regression_Results.csv`, `Bulk Tests/Julia/Jl_Regression_Results` `Bulk Tests/python/Py_Regression_Results.csv`

4.15.1 Full Parameter Name Divergence Table

Parameter	R CSV	Julia CSV	Python CSV
Intercept	`(Intercept)`	`(Intercept)`	`Intercept`
Holes	`Holes`	`Holes`	`Holes`
Urban County	`factor(county_type)Urban`	`county_type: Urban`	`C(county_type)[T.Urban]`

Parameter	R CSV	Julia CSV	Python CSV
Row order	Intercept, Holes, Urban	Intercept, Holes, Urban	Intercept, Urban, Holes

Python drops parentheses from `Intercept` — a statsmodels convention; R and Julia are consistent. Row order differs: Python places Urban before Holes. Phase 6 `compute_grand_means()` must match by parameter name, not by row index.

4.15.2 Coefficient Comparison (from actual M=100 Bulk Tests CSVs)

Parameter	R	Julia	Python	Spread
Intercept	12.2292	12.2471	12.2822	0.053 (0.4%)
Holes	0.05251	0.04764	0.04740	0.00511 (10.3%)
Urban County	4.00145	4.15774	4.17199	0.17054 (4.1%)

No order-of-magnitude divergence. R's Holes coefficient is ~10% higher than Julia/Python, attributable to R using `final_acreage` (OSM + Tigris acreage) while Julia/Python use `osm_acreage` (OSM-only). Same data asymmetry documented in Phase 2D.

4.15.3 Standard Error and FMI Comparison

Parameter	R SE	Jl SE	Py SE	R FMI	Jl FMI	Py FMI
Intercept	0.04141	0.03884	0.03856	0.014	0.040	0.079
Holes	0.002653	0.002392	0.002363	0.047	0.019	0.032
Urban	0.025054	0.020123	0.022576	0.339	0.095	0.392

Urban FMI is high in R (0.339) and Python (0.392) but low in Julia (0.095). Urban `df_adj` is very low: Python=25.9, R=34.8, Julia=426.8. High between-imputation variance for the Urban coefficient under R's Random Forest MICE and Python's LightGBM MICE backends. Julia's Mice.jl backend produces substantially lower between-imputation variance for this coefficient.

4.15.4 Checklist Items

1. **Parameter names**: Not fully consistent — three distinct naming conventions, one additional Python-specific divergence (no parentheses on Intercept). No fix applied — language-native conventions.

2. **Coefficient magnitude**: In plausible range; no order-of-magnitude divergence.
3. **SE and FMI columns**: Present and populated in all three CSVs.
4. **p_value numeric**: All values are float (not string); all parameters *** in all three.
5. **$0.944T plausibility**: $0.944T is the Phase 3 aggregate national opportunity cost from Rubin's pooled sum — it is NOT derived from Phase 4 regression coefficients. Phase 4 coefficients feed Phase 6 Forest Plots (^ comparison per language). The checklist item conflates the two. Phase 4 coefficients are directionally consistent with the economics (large positive Urban premium, positive Holes effect).

4.15.5 Critical Operational Observation

Canonical `Data/` output directories are empty. All three regression CSVs and model objects exist only in `Bulk Tests/{R,Julia,python}/` subdirectories. The master Phase 4 scripts (`Phase_4.R`, `Phase_4.jl`, `Phase_4.py`) write to: - `Phase 4 Econometric Modeling/Data/R/R_Regression_Results.csv` - `Phase 4 Econometric Modeling/Data/Julia/Jl_Regression_Results.csv` - `Phase 4 Econometric Modeling/Data/python/Py_Re`

None of these files currently exist. Phase 6 scripts reading from those paths will fail until the three master Phase 4 scripts are executed to completion. The Bulk Tests outputs are functionally correct and serve as confirmation that the pipeline runs; the master scripts need to be run to populate the canonical output locations.

5 Phase 5 Summary: Hawaii Micro-Case Study

5.1 Overview

Phase 5 conducted a micro-case study to validate the model's opportunity cost estimates against official county tax assessment data for golf courses in Hawaii. This provides empirical evidence of whether the model's Highest and Best Use (HBU) valuations align with real-world property tax assessments.

Phase 5 is organized in two tracks:

- **Phase 5a (Pilot):** A manual spot-check of 6 high-profile courses comparing model estimates to hand-retrieved assessment values.
- **Phase 5b (Full Pipeline):** A 6-step automated spatial pipeline — OSM polygon extraction, cadastral parcel intersection, economic validation via Rubin's Rules, TMK diagnostic merge, geographic breakdown, and zoning intersection analysis — implemented in R, Python, and Julia.

5.2 Methodology

5.2.1 Data Sources

1. **Phase 1 CSV:** `Py_Phase1_Baseline_Golf_Valuation.csv` — 16,297 courses with OSM acreage and baseline value per acre
2. **Phase 2 GeoPackage:** `Py_Phase2_OSM_Golf_Polygons.gpkg` — OSM-derived golf course polygons
3. **Honolulu Cadastral GeoPackage:** `All_Parcels_6378200148342636690.gpkg` — Honolulu parcel cadastre
4. **Honolulu Tax CSV:** `All_Parcels_-4613852522541990741.csv` — parcel-level tax/zone attributes
5. **Honolulu Zoning GeoPackage:** `Zoning_-2205419429161838665.gpkg` — Honolulu zoning layer (EPSG 3760, NAD83(HARN)/Hawaii zone 3, ftUS)
6. **Phase 3 Imputed Datasets (×100):** MICE-imputed economic datasets for Rubin's Rules pooling (M=100 per language)

5.2.2 Model Logic

Opportunity Cost is calculated as:

```
Opportunity_Cost = osm_acreage × Baseline_Value_Per_Acre
```

Where `Baseline_Value_Per_Acre` is determined by: - **Urban counties (RUCC 1–3):** FHFA Residential Land Price index - **Rural counties (RUCC 4–9):** USDA Agricultural Land Value

5.2.3 Phase 5b Pipeline Steps

Step	Description
1	OSM polygon extraction (Oahu bbox), Phase 1 point-in-polygon match rate, parcel reprojection
2	Cookie-cutter spatial intersection of OSM golf polygons over cadastral parcels → TMK list
3	MICE pooling (Rubin's Rules), spatial deduplication (500 m cap), economic validation table
4	Diagnostic only — verifies TMK format compatibility between Step 2 output and Honolulu CSV
5	Geographic concentration breakdown by TMK zone/district
6	Zoning intersection analysis — quantifies golf footprint by zone class and zone penetration rate

---

5.3 Results

5.3.1 Phase 5a: Pilot Course Comparison (Manual Spot-Check)

5.3.1.1 Hawaii Course Summary (All Islands, Phase 1 Filter)

Metric	Value
Total Courses (Hawaii state)	74
Average Opportunity Cost	$456,829,248
Total Opportunity Cost	$27,866,584,127

5.3.1.2 Summary by County

County	Courses	Total Opportunity Cost	Average Opportunity Cost
Honolulu	35	$25.1B	$866.2M
Hawaii (Big Island)	19	$26.7M	$1.7M
Maui	12	$2.7B	$299.5M
Kauai	8	$23.8M	$3.4M

5.3.1.3 Model vs. Official Assessment Comparison (6 Courses)

Course Name	County	Model Cost	Official Value	Gap Ratio
Turtle Bay Resort & Golf Club	Honolulu	$2.27B	$1.85B	1.23x
Waialae Country Club	Honolulu	$718.7M	$620.0M	1.16x
Kaanapali Golf Courses	Maui	$565.3M	$480.0M	1.18x
Wailea Golf Club	Maui	$255.1M	$210.0M	1.21x
Hualalai Golf Club	Hawaii	$295.1M	$175.0M	1.69x
Kohala Country Club	Hawaii	$625.0M	$420.0M	1.49x

Average model-to-assessed ratio: 1.33x — model estimates are on average 32.6% higher than official assessments.

5.3.2 Phase 5b: Full Pipeline Results (Automated, Honolulu County)

5.3.2.1 Step 1–3 Economic Validation

Metric	Value
Phase 1 baseline courses on Oahu	35
Phase 2 OSM golf polygons on Oahu	39
Direct point-to-polygon match rate	31.4% (11/35)
Unique TMKs intersecting golf polygons	1,073
OSM-derived legal footprint (live from Step 2)	8,564.23 acres
Deduplicated Oahu courses (500 m spatial cap)	33
Pooled opportunity cost — $\bar{q}$	**$25.400B**
Standard error	$1.397B
95% CI	$22.663B – $28.137B

5.3.2.2 Step 5 Geographic Concentration

Zone	District	Parcel Count	% of Parcels
9	Ewa (Kapolei/Pearl City)	677	63.2%
3	Honolulu (Anomalies)	169	15.8%
1	Honolulu (Urban Core)	103	9.6%
4	Koolaupoko (Kailua/Kaneohe)	55	5.1%
6	Waialua (North Shore)	38	3.5%
8	Waianae (West)	14	1.3%
7	Wahiawa (Central)	11	1.0%
2	Honolulu (East/Anomalies)	5	0.5%
—	**TOTAL**	**1,072**	**100.0%**

Note: 1,073 TMKs yield 1,072 parent parcel rows; one TMK has no matching parent record in the cadastral CSV after CPR sub-parcel filtering.

5.3.2.3 Step 6 Zoning Intersection (Python/Julia canonical; 6,066.2 acres)

Zone Class	Description	Acres	% of Golf Total
P-2	General Preservation District	3,209.4	52.9%
F-1	Federal and Military Preservation	1,002.0	16.5%
P-1	Restricted Preservation District	744.6	12.3%
AG-2	General Agriculture District	621.8	10.3%
AG-1	Restricted Agriculture District	213.8	3.5%
Resort	Resort District	130.4	2.2%
C	Country District	60.9	1.0%
R-5	R-5 Residential District	33.7	0.6%
Other (11 classes)	B-1, A-2, BMX-3, R-7.5, R-10, A-1, B-2, R-20, R-3.5, I-2, IMX-1	~50.1	~0.8%
—	**TOTAL**	**6,066.2**	**100.0%**

Preservation + Federal share: P-1 + P-2 + F-1 = 4,956 acres = **81.7%** of all Oahu golf land sits within Preservation or Federal/Military zones.

5.3.2.4 Step 6 Zone Penetration (what share of each Honolulu zone class is golf)

Zone Class	Description	Zone Total (ac)	Golf (ac)	% Golf
Resort	Resort District	513.7	130.4	25.4%
P-2	General Preservation	17,259.6	3,209.4	18.6%
B-1	Neighborhood Business	399.1	13.2	3.3%
F-1	Federal/Military Preservation	38,561.0	1,002.0	2.6%
C	Country District	3,254.3	60.9	1.9%
AG-2	General Agriculture	41,759.9	621.8	1.5%
R-20	R-20 Residential	521.2	3.1	0.6%
P-1	Restricted Preservation	157,429.3	744.6	0.5%
AG-1	Restricted Agriculture	63,462.2	213.8	0.3%

Cross-language note: R master reports P-1 acreage as 523.5 acres (total: ~5,845 acres); Python and Julia (standalone step and master) report 744.6 acres (total: ~6,066 acres). All other zone classes agree to <0.1 acres. The discrepancy likely reflects minor boundary-handling differences in `sf::st_intersection` vs. GDAL/Shapely at P-1 polygon edges. Python/Julia results are used as canonical.

5.4 Key Findings

1. **Systematic Overestimation (Phase 5a):** The model's HBU valuations are consistently higher than official tax assessments, with an average 1.33x premium (32.6% higher on average). This gap is largest for Big Island rural courses (1.49x–1.69x) and narrowest for Honolulu urban courses (1.16x–1.23x).
2. **Preservation-Dominated Footprint (Step 6):** 81.7% of Oahu golf land sits within Preservation or Federal/Military zones — areas where residential redevelopment would face the highest regulatory barriers. This challenges the HBU assumption for a large share of the golf footprint.
3. **Resort Zone Penetration (Step 6):** Golf occupies 25.4% of all Resort-zoned land on Oahu — the highest penetration rate of any zone class — and 18.6% of P-2 General Preservation land, underscoring golf's outsized role in how resort and preserved land is currently used.

4. **Geographic Concentration (Step 5):** 63.2% of golf parcels are in the Ewa district (Zone 9, Kapolei/Pearl City area), reflecting Oahu's post-statehood suburban golf development corridor rather than the urban core.
5. **Point-to-Polygon Representational Error:** Only 31.4% of Phase 1 course points fall directly within a Phase 2 OSM polygon, confirming significant coordinate-to-boundary mismatch across source datasets.
6. **Turtle Bay Case Study:** Turtle Bay Resort shows a 23% model premium ($2.27B vs. $1.85B assessed), demonstrating that the HBU framework captures potential residential development value not fully reflected in current tax assessments.

5.5 Limitations

1. **Assessment Timing:** Official values are from 2022; model uses 2022 baseline values but may reflect different market conditions.
2. **Zoning Regulatory Approval:** Classification in a Preservation or Agricultural zone indicates restriction, not an absolute prohibition. HBU analysis must treat such parcels separately.
3. **Assessment Practices:** County assessment practices vary; some counties use different valuation methodologies.
4. **Phase 5a Sample Size:** Only 6 high-profile courses were manually verified in the pilot; broader validation requires more data.
5. **Tax Exemptions:** Some courses may have tax-exempt status or special assessments not captured in the data.
6. **Cross-Language P-1 Discrepancy:** R `st_intersection` yields ~221 fewer acres in P-1 than Python/Julia; root cause unconfirmed (likely edge-case geometry handling).

5.6 Recommendations

1. **Zoning-Adjusted HBU:** Separate golf parcels in Preservation/Federal zones from those in residential/resort zones before applying HBU valuations; the opportunity cost for preservation-zone land is effectively zero under current zoning.
2. **Refine Baseline Values:** Consider adjusting baseline values to account for actual tax assessment ratios in each county.
3. **Expand Validation:** Include more courses in the Phase 5a validation sample for robustness.

4. **County-Specific Adjustments:** Develop county-specific valuation adjustments based on assessment ratio studies.
5. **Temporal Alignment:** Ensure baseline values and assessment values are from the same valuation date.

5.7 Implementation Notes

5.7.1 Output File Conventions

Language	Output Dir	Filename Prefix	Example
R master	`Data/R/`	none	`Phase5_Step6_Zoning_Percentag`
Python master	`Data/Python/`	`Py_`	`Py_Phase5_Step6_Zoning_Percen`
Julia master	`Data/Julia/`	`Jl_`	`Jl_Phase5_Step6_Zoning_Percen`
Bulk step scripts	`Bulk Tests/<lang>/`	none	`Phase5_Step6_Zoning_Percentag`

Python intermediate GeoPackages (`Target_Golf_Polygons.gpkg`, `Honolulu_Parcels_Reprojected.gpkg`) and `Target_Golf_Parcels_List.csv` remain in `Bulk Tests/python/` because the individual bulk step scripts read from there.

5.7.2 Scripts Inventory

Script	Language	Role
`Phase_5.R`	R	Master pipeline — 6-step end-to-end
`Phase_5.py`	Python	Master pipeline — 6-step end-to-end
`Phase_5.jl`	Julia	Master pipeline — 6-step end-to-end
`Bulk Tests/R/Step1_Data_Acquisition.R`	R	
`Bulk Tests/R/Step2_Parcel_Intersection.R`	R	
`Bulk Tests/R/Step3_Final_Comparison.R`	R	Rubin's Rules pooling

Script	Language	Role
`Bulk Tests/R/Step4_Offical_Tax_Merge.R`	R	Diagnostic merge
`Bulk Tests/R/Step5_Gap_Analysis.R`	R	Geographic breakdown
`Bulk Tests/R/Step6_Zoning_Intersection.R`	R	Zoning intersection analysis
`Bulk Tests/python/Step1_Data_Acquisition.py`	Python	
`Bulk Tests/python/Step2_Parcel_Intersection.py`	Python	
`Bulk Tests/python/Step3_Final_Comparison.py`	Python	
`Bulk Tests/python/Step6_Zoning_Intersection.py`	Python	Zoning intersection analysis
`Bulk Tests/Julia/Step1_Data_Acquisition.jl`	Julia	OSM polygon extraction + parcel reprojection
`Bulk Tests/Julia/Step2_Parcel_Intersection.jl`	Julia	Cookie-cutter TMK extraction
`Bulk Tests/Julia/Step3_Final_Comparison.jl`	Julia	Rubin's Rules pooling + economic validation
`Bulk Tests/Julia/Step4_Offical_Tax_Merge.jl`	Julia	Diagnostic merge (console output only)
`Bulk Tests/Julia/Step5_Gap_Analysis.jl`	Julia	Geographic concentration breakdown
`Bulk Tests/Julia/Step6_Zoning_Intersection.jl`	Julia	Zoning intersection analysis

5.7.3 Dependencies

Language	Key Packages
R	tidyverse, sf, this.path
Python	geopandas, pandas, numpy, pygris, pathlib
Julia	ArchGDAL, GeoDataFrames, DataFrames, CSV, Printf

5.8 Code Standardization

All Phase 5 scripts were standardized for naming consistency, path resolution, and structural conventions. No logic, formulas, spatial operation parameters, or filter thresholds were changed.

5.8.1 Four-Section Structure

Every script follows the same section order:

```
# === 1. LIBRARIES ===      (R, Python)  /  # === 1. USING ===  (Julia)
# === 2. GLOBALS & PATHS ===
# === 3. FUNCTIONS ===
# === 4. EXECUTION ===
```

Sections with no content use `# (none)` as the body.

5.8.2 Variable Renaming

Old name	New name	Applies to
`gdf_osm`, `osm_sf`	`osm_golf_geo` / `osm_golf_sf`	all languages
`gdf_parcels`, `parcels_sf`	`parcels_geo` / `parcels_sf`	all languages
`all_oahu_osm`	`oahu_golf_sf`	R
`phase1_df`	`baseline_df`	R
`phase1_sf`	`oahu_baseline_sf`	R
`oahu_boundary`	`oahu_boundary_sf`	R
`turtle_bay_polygon`, `turtle_bay_reprojected`	`target_golf_geo`	Python, Julia
`gdf_turtle_bay`	`target_golf_geo`	Python, Julia
`result` (intersection)	`parcel_intersection_sf` / `parcel_intersection_geo`	all
`result_df`	`tmk_df`	all
`gdf_turtle_bay`, `gdf_parcels`	`target_golf_geo`, `parcels_geo`	Python, Julia
`df` (comparison)	`comparison_df`	Python, Julia
`course_name` (literal constant)	`TARGET_COURSE`	Python, Julia

Old name	New name	Applies to
`script_dir`, `root_dir`	`SCRIPT_DIR`, `WORK_DIR`	R master
`Q_bar`, `V_W`, `V_B`, `V_T`, `SE`, `CI_lo`, `CI_hi`	`q_bar`, `v_w`, `v_b`, `v_t`, `se`, `ci_lo`, `ci_hi`	R (Step 3, Phase_5.R)
`osm_polys`	`osm_polys_sf`	R
`fmt_currency()` (unused)	removed	Julia Step 3
`geo_summary` (Step 5)	`zone_summary_z6`, `zone_penetration_z6`	R, Julia masters (Step 6, avoids collision)

5.8.3 Path Resolution

Language	Anchor	Example
R	`this.path::this.dir()`	`SCRIPT_DIR <- this.path::this.dir()`
Python	`pathlib.Path(__file__).parent`	`SCRIPT_DIR = Path(__file__).parent`
Julia	`@__DIR__`	`const SCRIPT_DIR = @__DIR__`

Bulk test scripts (paths from `Bulk Tests/<lang>/`): - 3 levels up → `WORK_DIR` (= `2 - Work/`) - 2 levels up → `PHASE5_DIR` (= `Phase 5 The Hawaii Micro-Case Study/`)

Master scripts (paths from `Phase 5 The Hawaii Micro-Case Study/`): - 1 level up → `WORK_DIR` - Outputs to `SCRIPT_DIR/Data/<Language>/`

5.8.4 `[METHODOLOGY]` Flags

Operation	Tag
`st_read()` / `GeoDataFrames.read()` / `gpd.read_file()`	spatial read of source layer
`st_write()` / `GeoDataFrames.write()` / `.to_file()`	persist for next step
`st_transform()` / `ArchGDAL.reproject()` / `.to_crs()`	CRS alignment
`st_filter()`	spatial subset to Honolulu county
`st_intersects()`	Phase 1 point-to-polygon match rate
`st_intersection()` / `gpd.overlay(how='intersection')` / `ArchGDAL.intersection`	cookie-cutter parcel or zoning intersection
`st_as_sf()`	tabular-to-spatial conversion

Operation	Tag
`st_nearest_feature()`	Phase 1 point → nearest OSM polygon assignment
`st_distance()`	500 m nearest-neighbor cap
`st_area()` / `ArchGDAL.geomarea()` / `.geometry.area`	area computation
Bounding box filter	21.2–21.9°N, −158.5 to −157.6°W — Oahu geographic filter

5.8.5 Language-Specific Conventions

- **R:** `library(dplyr)` + `library(readr)` + `library(stringr)` consolidated to `library(tidyverse)` in Steps 4, 5, and Phase_5.R. All `%>%` replaced with `|>`. `1:n` loops replaced with `seq_len(n)`.
- **Python:** All scripts wrapped with `def main(): ... if __name__ == "__main__": main()` guard. `import os` replaced with `from pathlib import Path`. `exit(1)` replaced with `raise SystemExit(1)`.
- **Julia:** All scripts wrapped with `function main() ... end` + `if abspath(PROGRAM_FILE) == @__FILE__ main() end` guard. All module-level variables declared `const`. Step 6 uses the `createcoordtrans` do-block pattern for CRS reprojection.

5.8.6 Bugs Flagged (Not Fixed)

1. **Step4 `else if` parenthesis error** — `all(nchar(tmk_df$TMK_clean)) == 9` should be `all(nchar(tmk_df$TMK_clean) == 9)`. Flagged with `[REVIEW NEEDED]`.
2. **Julia Step1/Step2 path inconsistency** — original Step 2 used `joinpath(@__DIR__, "..", "Data", "Julia")` (1 level up, `Bulk Tests/Data/Julia/`) while Step 1 used 2 levels up (`Phase5/Data/Julia/`). Standardized both to `PHASE5_DIR/Data/Julia/`.

5.8.7 Fixes Applied

1. **`"..." typo in WORK_DIR`** — Steps 4 and 5 had `file.path(SCRIPT_DIR, "..", "..", "...")` with a literal three-dot string. Corrected to `file.path(SCRIPT_DIR, "..", "..", "..")`.
2. `ROOT_DIR` → `WORK_DIR` in Step 1 — renamed for cross-script consistency.
3. **Duplicate "Total Official Assessed Value" row** in Step 3 bulk test — removed.
4. **Python master output paths** — `Phase_5.py` was writing all CSVs to `Bulk Tests/python/`; corrected to `Data/Python/Py_*.csv` to match the Phase 1/2 convention. Intermediate GeoPackages remain in `Bulk Tests/python/`.

5. **REGRESSION_CSV dead code removed (Julia & Python masters)** — Both `Phase_5.jl` and `Phase_5.py` declared a `Jl_Regression_Results.csv` / `Py_Regression_Results.csv` constant and ran existence checks on it, but never loaded it. Phase 5 computes opportunity cost directly as `osm_acreage × Baseline_Value_Per_Acre` from Phase 3 imputed datasets; Phase 4 regression outputs play no role. The declaration, header comment, and existence check were removed from both master scripts.
6. **Python `# === 1. LIBRARIES ===` section header** — `Phase_5.py` used the non-standard heading `# === 1. IMPORTS ===`; corrected to `# === 1. LIBRARIES ===` per CLAUDE.md convention (applies to all Phase master scripts).
7. **Python memory management in imputed dataset loop** — `run_step3()` loaded 100 CSVs without releasing memory between iterations. Added `del df_i; gc.collect()` after extracting the Oahu subset in each loop iteration; added `import gc` to Section 1.
8. **Python `[METHODOLOGY]` tags in `run_step6()`** — Two `gpd.read_file()` calls (golf polygons and zoning layer) were missing the required `# [METHODOLOGY]` tag. Added.
9. **Python `OSM_DERIVED_ACRES = 8342.28` replaced with live computation** — The hardcoded module-level constant reflected an earlier run and became stale. `run_step2()` already computed `total_acres` from the Step 2 intersection geometry; it now returns that value. `run_step3()` accepts it as a parameter (`osm_derived_acres`), and `main()` captures and forwards it. See also *Acreage Discrepancy Note* below.
10. **Python `Zone_Code` float string bug in `run_step5()`** — The `Zone` column from the Honolulu cadastral CSV reads as `float64`; `.astype(str)` produced `"9.0"` rather than `"9"`, causing all `DISTRICT_MAP` lookups to fall through to the `"Zone 9.0"` fallback. Added `dropna(subset=["Zone"])` (parallel to Julia's `dropmissing!(geo_merged, :Zone)`) and changed the cast to `.astype(int).astype(str)`. District names now resolve correctly (e.g. `"Ewa (Kapolei/Pearl City)"`).

5.9 Julia Pipeline Implementation

Date Completed: May 1, 2026 All Julia bulk-test step scripts (Steps 1–6) are debugged and fully functional. The standalone master script `Phase_5.jl` runs all steps end-to-end in memory without invoking daughter scripts.

5.9.1 Data Flow (Standalone Master)

```
Phase 1 CSV                          Step 1: point-in-polygon rate
Phase 2 GPKG   oahu_golf_geo
Parcels GPKG   parcels_geo (reprojected)        Step 2: spatial intersection

                     unique_tmks (1,073)          Step 3: economic validation
```

```
Phase 3 CSVs (×5)                                    (Rubin's Rules + dedup)
Phase 4 CSV

                      unique_tmks                 Step 5: geographic breakdown
Honolulu tax CSV
                                                                Step 6: zoning intersection
Zoning GPKG    (reprojected to EPSG 5070)          golf × zoning fragments → CSVs
oahu_golf_geo
```

5.9.2 Output Files (Standalone Master)

File	Description
`Data/Julia/Jl_Phase5_Oahu_Comparison.csv`	Comparison validation table (pooled opportunity cost, CI, acreage)
`Data/Julia/Jl_Phase5_Geographic_Breakdown.csv`	Zone-level opportunity costs and share of total
`Data/Julia/Jl_Phase5_Step6_Zoning_Intersections.csv`	Golf fragments by zoning class (acres, % of total)
`Data/Julia/Jl_Phase5_Step6_Zone_Shares.csv`	% of each zoning class occupied by golf

5.9.3 Bugs Fixed in Julia Step Scripts

5.9.3.1 Step 3 — `MethodError: no method matching reproject`

- **Root cause:** `reproject_geom` called `ArchGDAL.reproject(geom, ISpatialRef, ISpatialRef)`, which does not exist in this version of ArchGDAL.jl.
- **Fix:** Replaced with the `createcoordtrans` do-block + `transform!` pattern:

```
function reproject_geom(geom, src_crs, tgt_crs)
    ArchGDAL.createcoordtrans(src_crs, tgt_crs) do t
        ArchGDAL.transform!(geom, t)
        geom
    end
end
```

5.9.3.2 Step 3 — Axis-swap bug (`importEPSG(4326)`)

- **Root cause:** `ArchGDAL.importEPSG(4326)` in GDAL 3.x uses official lat/lon axis order, silently swapping coordinate axes.
- **Fix:** Changed to `ArchGDAL.importPROJ4("+proj=longlat +datum=WGS84 +no_defs")` (guarantees lon/lat order). Used in both step scripts and master.

5.9.3.3 Steps 4 & 5 — `ArgumentError: Duplicate variable names: :TMK`

- **Root cause:** Both the Step 2 TMK list and the Honolulu cadastral CSV have a column named `"TMK"`. `innerjoin` on `:TMK_clean` collides on the shared `:TMK` column.
- **Fix:** Added `makeunique = true` to every `innerjoin` where a Step 2 TMK list is joined against the Honolulu CSV.

5.9.3.4 Step 5 — 83.6% of rows showing missing district zone

- **Root cause:** Honolulu cadastral CSV contains CPR sub-parcel records that share a parent TMK but carry `missing` in the `Zone` column, causing a 6.1× row multiplier.
- **Fix:** Added `dropmissing!(merged_data, :Zone)` immediately after the join. Row count dropped from 6,556 to 1,072.

5.9.3.5 Step 6 — `column name :geometry not found`

- **Root cause:** The Honolulu zoning GeoPackage stores geometry in the column `SHAPE` (ArcGIS convention); GeoDataFrames does not normalize this to `geometry`.
- **Fix:** Changed `reproject_geoms(zoning_gdf.geometry, ...)` to `reproject_geoms(zoning_gdf.SHAPE, ...)`.

5.9.3.6 Step 6 — `MethodError: no method matching createcoordtrans(::ISpatialRef, ::ISpatialRef)`

- **Root cause:** This version of ArchGDAL.jl requires `createcoordtrans` to take a `Function` as its first argument (resource-management/do-block pattern); a direct two-argument call is unsupported.
- **Fix:** Rewrote `reproject_geoms` as a do-block accumulator:

```
function reproject_geoms(geoms, src_epsg::Int, dst_epsg::Int)
    src_sr = ArchGDAL.importEPSG(src_epsg)
    dst_sr = ArchGDAL.importEPSG(dst_epsg)
    result = ArchGDAL.IGeometry[]
    ArchGDAL.createcoordtrans(src_sr, dst_sr) do coord_tf
        for g_orig in geoms
            g = ArchGDAL.clone(g_orig)
            ArchGDAL.transform!(g, coord_tf)
            push!(result, g)
        end
    end
    return result
end
```

5.9.4 Acreage Discrepancy Note

The daughter script `Step3_Final_Comparison.jl` uses a hardcoded constant `OSM_DERIVED_ACRES = 8342.28`. The standalone `Phase_5.jl` computes acreage live from the Step 2 spatial intersection geometry, yielding **8,564.23 acres**. `Phase_5.py` previously carried the same stale constant (`OSM_DERIVED_ACRES = 8342.28` in Section 2); this has been corrected — `run_step2()` now returns `total_acres` computed live, and `run_step3()` accepts it as a parameter. The live-computed value (**8,564.23 acres**) is authoritative; the hardcoded constant in the Julia daughter script `Step3_Final_Comparison.jl` remains stale but that script is not part of the master pipeline.

5.10 Cross-Language Consistency Notes (Post-Audit)

5.10.1 Oahu OSM Polygon Count

Each master script reads its own language-prefixed Phase 2 GeoPackage (`R_Phase2_OSM_Golf_Polygons.gpkg`, `Jl_Phase2_OSM_Golf_Polygons.gpkg`, `Py_Phase2_OSM_Golf_Polygons.gpkg`). The spatial filter to Oahu's bounding box produces a one-course difference:

Language	Oahu OSM polygons
R	38
Julia	39
Python	39

This difference is traceable to minor variation in the three Phase 2 runs (different polygon parse order, potential edge-case geometry differences) and is within the expected cross-language Phase 2 spread. It does not affect the economic validation conclusions.

5.10.2 Geographic Breakdown (Step 5)

All three languages produce identical parcel counts and zone distributions (1,072 parcels; Zone 9 = 63.2%). Prior to the audit, Python displayed zone codes as `"9.0"` and district names as `"Zone 9.0"` due to float-to-string coercion. This has been corrected; all three languages now produce matching `Zone_Code` (integer string) and `District_Name` values.

5.10.3 37-Course Ewa District Figure

The figure "37 courses in Ewa District (Zone 9)" referenced in Phase 6 output cannot be confirmed from Phase 5 alone — Phase 5 Step 5 produces parcel-level zone breakdowns, not course-level counts by district. Verification requires cross-referencing Phase 6 Script 9 (Oahu spatial summary).

6 Phase 6 — Visualization

6.1 Overview

Phase 6 produces all publication-ready figures, maps, and LaTeX tables for the thesis. Scripts are organized under `Bulk/R/` and `Bulk/Julia/` and follow a shared four-section layout (`LIBRARIES`, `GLOBALS & PATHS`, `FUNCTIONS`, `EXECUTION`). All raster outputs target 300 DPI equivalent.

6.1.1 Language Assignment Strategy

After evaluating output quality across all three candidate languages, the following assignment has been adopted:

Language	Scope
R	All spatial/geographic map outputs — national choropleths, county maps, Oahu micro-maps, bivariate maps, and the Oahu opportunity cost map. R's `sf` + `ggplot2` + `tigris` stack consistently produces higher-quality cartographic output than the Julia GeoMakie pipeline.
Julia	All non-spatial outputs — statistical plots (forest, density, marginal effects, raincloud) and LaTeX table generation. CairoMakie produces clean, publication-quality figures for these chart types.
Python	Not used. Python performs poorly for both data visualization (charts) and physical/cartographic mapping relative to R and Julia in this context.

6.1.2 Last Verified Run — May 6, 2026

`Phase_6.R` executed successfully (exit code 0). All outputs confirmed written.

Metric	Value
Tri-Language Grand Mean — National Total	**$0.944T**

Metric	Value
Observed-only baseline	\$0.788T (82.5% of MICE-pooled)
States covered	51
Counties covered	2,890
Oahu pooled OC (M=100 R draws)	\$31.197B across 37 courses
OLS coefficients	=12.225, _holes=0.053, _urban=4.002

Top 5 states (Grand Mean): CA \$293.70B · FL \$111.95B · NY \$53.91B · TX \$38.61B · HI \$35.61B
Top 5 counties: Los Angeles \$49.73B · Orange \$38.47B · Santa Clara \$33.80B · San Diego \$30.31B · Honolulu \$29.69B

Script 15 (resolved May 2026, Part 6C): Log-residual range corrected from `[NaN, NaN]` (all-gray render) to `[-2.809, 4.437]`. Dollar-residual corrected from `$-1.766e12` absurd magnitudes to `[$-1.30B, $45.86B]`. See Part 6C in the Structural Audit Log and the Script 15 section for full fix details.

6.1.3 Structural Audit Log

Phase_6.jl — Audited May 8, 2026 (Checklist Point 26)
Full read of all 1,843 lines across 7 modules. Three categories of issues identified and corrected:

1. **Path case inconsistency — 7 locations fixed.** Mod_5, Mod_6, Mod_10, Mod_11, and Mod_12 used `"Python"` (capital-P) in Phase 3 and Phase 4 data directory path components. The authoritative directory file specifies `python` (lowercase). Fixed at lines 143, 264, 306, 322, 639, 970, and 1242. Mod_13 and Mod_14 were already correct.
2. **Mislabeled `[METHODOLOGY]` flags — 2 removed.** Comments reading `# [METHODOLOGY] Spatial read of Phase 1 Jl baseline` appeared above plain `CSV.read()` calls in Mod_11 (line 1151) and Mod_12 (line 1428). `CSV.read()` does not qualify as a spatial read per CLAUDE.md; both comments removed.
3. **Top-level execution block not wrapped in `main()` — 1 fix.** The `Threads.@spawn` / `fetch` / `GC.gc()` block at lines 1817–1842 existed at file top level. CLAUDE.md requires all Julia logic inside `main()`. Block wrapped in `function main() ... end` with `main()` called at the bottom of the file.

All other structural requirements passed: `@__DIR__` path resolution correct, Rubin's Rules independent per language (M=100 each), Grand Mean as arithmetic mean of three pooled

vectors, memory-safe loops with `df = nothing; GC.gc()` throughout, 20 density PNGs with correct `5.211-5.245` naming, four-section layout in all modules, no `log(acreage)` anywhere.

Phase_6.R — Audited May 8, 2026 (Checklist Point 27)
Full read of all ~2,280 lines across `compute_grand_means()` and seven `run_X_()` functions. Six categories of issues identified and corrected:

1. **Missing top-level Section 2 & 3 headers — added.** The file-level four-section structure lacked `# === 2. GLOBALS & PATHS ===` and `# === 3. FUNCTIONS ===` headers between the LIBRARIES block and `compute_grand_means()`. Both added with the required two blank lines between sections.
2. **Execution code outside Section 4 — moved.** `grand_means <- compute_grand_means()` and `plan(sequential)` appeared at file top level between function definitions (after `compute_grand_means()`, before `run_1_Macro_Maps()`). Moved into the `# === 4. EXECUTION ===` block where they now run before all `run_X_()` calls.
3. **Section 4 header missing number — fixed.** `# === EXECUTION ===` corrected to `# === 4. EXECUTION ===`.
4. **Missing `[METHODOLOGY]` flag on Rubin's q_bar in `pool_oahu_oc()` — added.** The `bind_rows() |> group_by() |> summarise(pooled_opp_cost = mean(...))` block inside the nested `pool_oahu_oc()` helper (Script 9) lacked a `[METHODOLOGY]` comment. Added above the summarise call.
5. **Stale loop variable in Script 1 coverage calculation — fixed.** `run_1_Macro_Maps()` Step 5 divided observed totals by `pooled_state$pooled_opp_cost`, which held the Julia pool after the loop. Fixed to reference `grand_means$state$GrandMean$pooled_opp_cost` directly.
6. **Stale loop variable in Script 2 coverage calculation — fixed.** Same bug in `run_2_County_Map()` Step 5 (`pooled_county` → `grand_means$county$GrandMean$pooled_opp_cost`).

All other structural requirements passed: `this.path::this.dir()` path resolution correct throughout, Rubin's Rules independent per language (M=100 each) in all pooling functions, Grand Mean used for all choropleth values, `rm()/gc()` inside every sequential dataset-loading loop, furrr parallel loops use pipeline pattern (no intermediate variables), only R_-prefixed CSVs read (Py_/Jl_ reads in master's tri-language aggregation context are correctly authorized), ObservedOnly variants present for Scripts 1, 2, 7, 9, Script 8 shows Py/R/Jl side-by-side, no `log(acreage)` anywhere, `[METHODOLOGY]` flags on all `st_read/st_transform/st_join`/pooling blocks, all constants in ALL_CAPS in Section 2.

Phase_6.R — Spot-Check May 9, 2026 (Checklist Part 6A)
Targeted re-verification of all 9 items fixed or confirmed during Point 27. Zero regressions found.

1. Four-section headers (`# === 1. LIBRARIES ===` through `# === 4. EXECUTION ===`) all present with two blank lines between — confirmed.
2. `compute_grand_means()` and all seven `run_X_()` functions reside in the file-level Section 3 — confirmed.
3. `grand_means <- compute_grand_means()` is at line 2274, inside the top-level `# === 4. EXECUTION ===` block — confirmed (not between function definitions).
4. `plan(sequential)` is at line 2275, immediately after the grand_means assignment — confirmed.
5. Script 1 Step 5 coverage calculation references `grand_means$state$GrandMean$pooled_opp_cost` (line 377) — confirmed, stale loop variable fix held.
6. Script 2 Step 5 coverage calculation references `grand_means$county$GrandMean$pooled_opp_cost` (line 620) — confirmed, stale loop variable fix held.
7. `# [METHODOLOGY]` tag on `pool_oahu_oc()` Rubin's q_bar block (line 1663) — confirmed.
8. No `log(acreage)` string anywhere — grep confirmed.
9. All `library()` calls in Section 1 only (lines 17–29) — grep confirmed, none elsewhere.

Bonus closure: Script 9 `POLYGONS_GPKG` (line 1586) resolves to `Phase 5 The Hawaii Micro-Case Study/Data/R/Target_Golf_Polygons.gpkg`, matching Phase 5 R write target exactly. Phase 5 → Phase 6 Script 9 handoff confirmed intact (closes open 5D item).

Phase_6.jl — Spot-Check May 9, 2026 (Checklist Part 6B)
Targeted re-verification of all 6 items fixed during Point 26. Zero regressions found.

1. `"python"` (lowercase) confirmed at all 7 fixed path-string locations: Mod_5 line 143 (lang tuple dir_name), Mod_6 line 264 (Phase 4 py reg path), Mod_10 lines 306 and 322 (Phase 4 reg + Phase 3 MICE dir), Mod_11 line 639 (Phase 3 dir), Mod_12 line 970 (Phase 3 dir), Mod_13 line 1241 (Phase 3 dir). Mod_14 correct as before (lines 1506, 1677).
2. No `# [METHODOLOGY] Spatial read` comment above any `CSV.read()` call — grep confirmed (both Mod_11 ~1151 and Mod_12 ~1428 removals held).
3. `function main() ... end` present in all 7 modules (Mod_5 through Mod_14) and in the top-level dispatcher block. `main()` call confirmed at file line 1844.
4. No `Plasma.jl` reference anywhere — grep confirmed.
5. No `log(acreage)` string anywhere — grep confirmed.
6. No M=300 single-pool anywhere — grep confirmed. All Rubin's Rules blocks pool independently at M=100 per language.

Minor observation: file-level comment header (line 7) reads `Data/Python/` (capital P) while the actual runtime path uses `"python"` (lowercase). Comment-only discrepancy; no runtime impact.

Phase 6 Integration Check — May 9, 2026 (Checklist Part 6C)
Cross-script integration audit: path routing, filename convention, output directory structure, and cross-language reads. 1 fix, 4 observations.

1. **PASS — `compute_grand_means()` tri-language path routing** (Phase_6.R lines 55, 85, 112): `PHASE3_DIR` (R), `PHASE3_PY_DIR` (python), `PHASE3_JL_DIR` (Julia) all constructed via `file.path(WORK_DIR, ...)`. All three language directories read correctly. No hardcoded absolute paths.
2. **OBS — Phase_6.jl reads from all three language directories by design**: Checklist item was overstated. Mod_11 (Lorenz, lines 1188/1194), Mod_13 (Counterfactual, lines 1454/1457), Mod_14 (Scatter, lines 1677–1678), and Mod_9 (Oahu OC, lines 878/894) all read from Py and R directories. All qualify as tri-language diagnostic visualization functions under the CLAUDE.md exception clause. Each language is processed independently; no cross-language statistical pooling occurs.
3. **OBS — $0.944T plausibility unverifiable without execution**: Output directories are empty (scripts not yet run). Per 4D tracker, $0.944T is the Phase 3 MICE national aggregate (acreage × FHFA), computed by `compute_grand_means()`, not a Phase 4 regression coefficient. Checklist phrasing was imprecise.
4. **OBS — Script 9 no per-language QA maps; Script 15 no ObservedOnly**: Script 9 (`run_9_Oahu_Opportunity_Cost_Map()`) produces only `9.141_GrandMean` and `9.101_ObservedOnly` — no `9.111_Julia`, `9.121_Python`, `9.131_R` QA variants. Script 15 produces two GrandMean maps (log + dollar) only — no ObservedOnly is possible without a separate observed-only regression pass.
5. **PASS — No filename conflicts**: Phase_6.R holds prefixes 1–4, 7, 9, 15; Phase_6.jl holds 5–6, 10–14. Sets are disjoint; no collision risk.
6. **FIX — `11_Lorenz_Curve_TriLanguage.png` violated 1.234 convention** (Phase_6.jl lines 19 and 979): renamed to `11.141_Lorenz_Curve_TriLanguage.png` in both the header comment and the `OUT_LORENZ` constant. All other Phase_6.jl output names follow the convention.
7. **OBS — Phase_6.jl header comment stale**: Line 4 says "scripts 5, 6, and 10" but file handles Mod_5 through Mod_14 (scripts 5, 6, 10–14). Output file list (lines 11–19) omitted `12.141_`, `13.141_`, `14.141_` outputs. Comment-only; no runtime impact. Output file index in this document corrected to match actual script output names.

Script 15 Residual Map Diagnostic — May 2026 (Checklist Part 6C)
Six formula and methodology bugs identified and corrected in standalone `Bulk/R/15_Residual_Map.R`, then ported one-for-one into `run_15_Residual_Map()` in `Phase_6.R`:

1. **Log-residual formula corrected.** Pre-patch formula `log(final_acreage) - predicted_log` was off by `log(BVPA)` 15.4 log-units. Corrected to `log(acreage × Baseline_Value_Per_Acre) - predicted_log`.
2. **Dollar-residual units error corrected.** Pre-patch subtracted dollars from acres then multiplied by $/acre, producing$ −1.766e12`magnitudes. Corrected to`(acreage × Baseline_Value_Per_Acre) - exp(predicted_log)‘ with both terms in dollars.
3. **`acreage > 1` filter added.** Guards against `log(0)` / `log(1e-15)` from near-zero MICE-imputed acreage values.
4. **FIPS-safe cross-language join.** `select(-any_of(c("FIPS", "County_Name",`

"State_Abbr", "Tigris_State_Abbr"))) before left_join(county_lookup, by = c("Longitude", "Latitude")). Drops native FIPS/county columns from Py/Jl (no-op for R). Fixes both the R FIPS-not-found error and the Julia State_Abbr column conflict.
5. **Holes range filter added.** filter(between(Holes, 9, 72)) guards against the 252-hole Phase 1 aggregate record causing exp(b_holes × 252) $3.7T explosion in the dollar-residual map.
6. **Verified output ranges (Grand Mean, M=300 total).** Log-residual: [−2.809, 4.437]. Dollar-residual: [$−1.30B, $45.86B]. Counties with residuals: 2,874 of 3,144.

CLAUDE.md updated: FIPS asymmetry corrected, Script 15 status marked Fixed.

FIPS-NA Diagnostic Audit — May 2026 (Checklist Part 6D)
Read-only national audit confirming 34 FIPS-NA courses (0.21% of 16,292) — consistent across all three language pipelines (R: 34, Py: 34, Jl: 34). Three questions resolved:

1. **Q1 — $4,952,600/acre anchor confirmed.** Exact match between thesis value and FHFA source (2024 - FHFA June 20 Land Prices.xlsx, sheet "Panel Counties", Year==2022, FIPS 15003). R's Phase 1 lookup reads the correct source value directly.
2. **Q2 — Root cause: cb=TRUE, resolution="20m" cartographic boundary simplification.** All 5 Hawaii FIPS-NA courses are 54–433 meters outside the 1:20,000,000 simplified polygon. Hawaii Kai (54.7m) and Mid-Pacific (140.4m) also resolve with cb=FALSE. Kahili and King Kamehameha Golf Club on Maui share identical coordinates at 352m from any polygon — likely a source data coordinate issue, not a pipeline error.
3. **Q3 — Thesis defensible.** 34 courses across 16 states (HI:5, CA:4, FL:4, WI:4, AL:3, MI:2, OR:2, SC:2; CT/MA/MD/ME/NY/OH/VA/WA: 1 each). Distribution is consistent with coastal/water-boundary simplification artifacts. The §5.4.2 footnote already covers the known Hawaii cases. No Phase 1 re-run required before defense.

Diagnostic outputs (read-only) in Phase 7 Documentation.../QA/: FIPS_NA_Audit.R, FIPS_NA_Audit_Report.md, FIPS_NA_Courses_R.csv (34 courses), FIPS_NA_State_Summary.csv.

Script 9b Rural-USDA Sensitivity — May 2026 (Checklist Part 6E)
New defense-only bulk script 9b_Oahu_OC_Map_Rural_USDA_Sensitivity.R written, standalone test run confirmed, and integrated into Phase_6.R as run_9b_Oahu_OC_Rural_USDA_Sensitivity(). CLAUDE.md updated with full methodology.

Reclassification keyed on ZONMAP_NO from Zoning_Map_Boundary.geojson (34 polygons, City & County of Honolulu Development Plan boundary layer): - Codes 1–14, 21–24 (urban/suburban core and Windward Oahu: Kualoa, Kaneohe, Kailua, Waimanalo) → FHFA ($4,952,600/ac) - Codes 15–20 (rural: Lualualei/Makaha, Makua/Kaena, Mokuleia/Wailua/Haleiwa, Kawailoa/Waialee, Kahuku/Laie, Hauula/Punaluu/Kaaawa) → USDA ($29,887/ac, read dynamically) - Code 0 — no golf courses present

FHFA normalization applied to ALL Oahu BVPA before any USDA override, correcting Hawaii Kai and Mid-Pacific (FIPS-NA courses whose BVPA was MICE-imputed

in Py/Jl rather than resolved from Phase 1). Zone assignment: `st_centroid` → `st_join(st_within)` → `st_nearest_feature` fallback (500m cap). Grand Mean: Rubin's Rules independently per language (M=100), arithmetic mean of three. Output: `9b.141_Oahu_OC_Map_Rural_USDA_Sensitivity_GrandMean.png` — standalone to `Bulk/R/output/`, master to `output/Final_Thesis_Figures/`.

Phase_6.jl Module 5 Library Fix — May 14, 2026
`Plots` package removed from `Mod_5_Econometric_Plots` `using` statement (was: `using CSV, CairoMakie, DataFrames, Printf, Colors, Plots`; now: `using CSV, CairoMakie, DataFrames, Printf, Colors`). `Plots.jl` and `CairoMakie.jl` export overlapping function names (`scatter!`, `lines!`, and others); loading both in the same module scope causes Julia binding-ambiguity errors at runtime. `Colors` is retained — required for the `colorant""` string macro used by the UHM color palette constants (`UHM_GREEN`, `UHM_GOLD`, etc.) added in the same editing pass. Remaining changes in that pass are cosmetic: subtitle color standardized to `#024731` (UHM Green), caption font sizes updated to 10pt, and `word_wrap = true` added to multi-line captions across all modules.

6.2 Master Scripts (Completed & Refactored)

The initial bulk scripts have been fully consolidated into two standalone master execution pipelines:

- **`Phase_6.R`** — A monolithic R script that aggregates and sequentially produces all geographic map outputs. It utilizes `furrr`, `future`, and `parallelly` to distribute MICE dataset loading across CPU cores. It also introduces **Tri-Language Grand Mean** logic: calculating the pooled mean of Python, R, and Julia imputed datasets to generate 4 comparative map outputs per core cartographic figure (Grand Mean, Python, R, and Julia variations).
- **`Phase_6.jl`** — A monolithic Julia script encapsulated via strict native modules to guarantee memory isolation. It sequentially produces all non-spatial figures and LaTeX tables. Core plots (e.g., Lorenz curves, Dumbbell gaps) have been refactored to plot tri-language color-coded series (Green: Python, Blue: R, Purple: Julia) to visually trace MICE variations.

6.2.1 General Naming Scheme & Output Routing

Standardize all visualization PNG outputs across the entire Phase 6 pipeline to follow the specific `1.234` logic format, where:

- **1** = Main script number (1 to 15)

- **.2** = Sub-category/part of the script
- **3** = Language identifier (1=Julia, 2=Python, 3=R, 4=GrandMean/All, 0=Observed-Only/No MICE)
- **4** = Sub-count / sequential index

The master scripts (`Phase_6.R` and `Phase_6.jl`) automatically route outputs into two distinct sub-directories based on the 3rd digit logic: - `output/Final_Thesis_Figures/`: Final assets intended for the LaTeX document (3rd digit `4` or `0`). - `output/QA_Verification/`: Internal visual verification files (3rd digit `1`, `2`, or `3`).

6.3 R Bulk Scripts

Nine R scripts have been completed under `Bulk/R/`. All use `this.path::this.dir()` for script-relative path resolution. Scripts 1, 2, 7, and 9 produce two map variants each: a MICE-pooled version (`.1`) and an observed-acreage-only version (`.2`).

6.3.1 Script 1 — `1_Macro_Maps.R`

Outputs: `output/1.1_National_Opportunity_Cost_Map.png` (14×9 in), `output/1.2_National_Opportunit` (14×9 in)

1.1 — MICE-pooled (4 Variations): State-level national choropleth. OC computed via MICE pooling. The script leverages the global Tri-Language Grand Mean calculation to automatically render and save four distinct variations of this map: `GrandMean`, `Python`, `R`, and `Julia`. Fill uses Viridis "plasma" on a linear scale. Alaska and Hawaii repositioned as insets via `tigris::shift_geometry()`. CRS: NAD83/Conus Albers (EPSG 5070).

1.2 — Observed-only: Identical map structure. OC restricted to courses with directly measured OSM polygon acreage (`acreage_source != "MICE_Target"` from Phase 2). Simple county-level sum; no pooling. States without observed-acreage courses rendered gray.

A shared `build_state_map()` function renders both outputs from a common ggplot template.

Key inputs: `R_Imputed_Dataset_{1..100}.csv`, `R_Phase1_Baseline_Golf_Valuation.csv`, `R_Phase2_Acreage_Matched_v2.csv`
R packages: `tidyverse`, `sf`, `tigris`, `scales`, `this.path`

6.3.2 Script 2 — `2_County_Map.R`

Outputs: `output/2.1_County_Opportunity_Cost_Map.png` (14 × 9 in), `output/2.2_County_Opportunity_C` (14 × 9 in)

2.1 — MICE-pooled (4 Variations): County-level choropleth aggregated to 5-digit FIPS. Utilizes the global Tri-Language Grand Mean calculation to render four map variants (`GrandMean`, `Python`, `R`, `Julia`). Fill uses Viridis "plasma" on a log scale to reveal geographic spread across the highly right-skewed county distribution. Counties without golf courses rendered in gray. Alaska and Hawaii shown as insets. CRS: EPSG 5070.

2.2 — Observed-only: Identical log choropleth restricted to courses with directly measured OSM acreage from Phase 2.

A shared `build_county_map()` function renders both outputs.

Key inputs: `R_Imputed_Dataset_{1..100}.csv`, `R_Phase1_Baseline_Golf_Valuation.csv`, `R_Phase2_Acreage_Matched_v2.csv`
R packages: `tidyverse`, `sf`, `tigris`, `scales`, `this.path`

6.3.3 Script 3 — `3_Oahu_TMK_Map.R`

Output: `output/3_Oahu_TMK_Concentration_Map.png`

High-resolution micro-map of Oahu rendering all 1,072 golf course TMK parcels. Ewa District (Zone 9) parcels are highlighted in orange-red (`#E05C14`); all other districts in dark gray. Island outline derived by dissolving all Honolulu parcels. North arrow and scale bar added via `ggspatial`. Projection: WGS 84 / UTM Zone 4N (EPSG 32604).

Key inputs: `Honolulu_Parcels_Reprojected.gpkg`, `Target_Golf_Parcels_List.csv`, `Phase5_Geographic_Breakdown.csv`
R packages: `tidyverse`, `sf`, `ggspatial`, `this.path`

6.3.4 Script 4 — `4_Oahu_Zoning_Map.R`

Output: `output/4_Oahu_Golf_Zoning_Map.png`

High-resolution micro-map of Oahu coloring the 1,072 golf course TMK parcels by their dominant zoning classification. Each parcel is assigned the `zone_class` of the Honolulu zoning polygon with the largest overlap area (`st_join(..., largest = TRUE)`). Island base and coastline rendered beneath. Projection: EPSG 32604.

Key inputs: `Honolulu_Parcels_Reprojected.gpkg`, `Target_Golf_Parcels_List.csv`, `Zoning_-2205419429161838665.gpkg`
R packages: `tidyverse`, `sf`, `ggspatial`, `this.path`

6.3.5 Script 5 — `5_Econometric_Plots.R`

Outputs: `output/5.1_Forest_Plot.png` (9×4 in), `output/5.2_MICE_Density_Plot.png` (10×6 in)

5.1 — Forest Plot: Displays Phase 4 regression coefficients (Intercept, Holes, Urban County) with 95% CIs (`Coef ± 1.96 × SE`), a vertical dashed reference line at zero, and human-readable parameter labels. Significance stars appended to labels where applicable.

5.2 — MICE Density Plot: Overlays the observed acreage distribution (courses with measured polygons, `acreage_source != "MICE_Target"`) against 100 independent MICE draws (imputed-only rows, identified via `semi_join` on Longitude × Latitude from Phase 2). Log x-axis. Distributional overlap validates the imputation.

Key inputs: `R_Regression_Results.csv`, `R_Phase2_Acreage_Matched_v2.csv`, `R_Imputed_Dataset_{1..10`
R packages: `tidyverse`, `scales`, `this.path`

6.3.6 Script 6 — `6_Advanced_Econometric_Plots.R`

Outputs: `output/6.1_Marginal_Effects_Dollar_Value.png` (7×6 in), `output/6.2_MICE_Raincloud_Diagr` (12×7 in)

6.1 — Marginal Effects Plot: Translates log-scale regression coefficients into predicted dollar opportunity costs for an average Rural vs. Urban course, with Holes fixed at the sample median. Prediction CIs computed via the delta method (diagonal variance only; covariance terms omitted, noted in caption). Point + errorbar + value label layers; `theme_classic()`.

6.2 — Raincloud Diagnostic: Compares observed acreage against all 100 MICE draws (imputed parcels only) using `ggdist::stat_halfeye` (half-violin) + `geom_boxplot` + jittered raw points. Acreage pre-transformed to log before plotting to avoid bandwidth selection issues; axis tick labels show original acre values. `theme_classic()`.

Key inputs: `R_Regression_Results.csv`, `R_Phase2_Acreage_Matched_v2.csv`, `R_Imputed_Dataset_{1..10`
R packages: `tidyverse`, `scales`, `ggdist`, `this.path`

6.3.7 Script 7 — `7_Bivariate_Econometric_Map.R`

Outputs: `output/7.1_Bivariate_Cost_vs_Density_Map.png` (14×9 in), `output/7.2_Bivariate_Cost_vs_I` (14×9 in)

Bivariate choropleth at the county level showing the joint distribution of total opportunity cost (x-dimension) and total golf course holes (y-dimension). Counties classified into a 3×3 grid via `biscale::bi_class(..., style = "quantile", dim = 3)`. Palette: `"DkViolet"`. 3×3 legend composed as a `cowplot` inset at position (0.72, 0.04). Alaska and Hawaii shown as insets.

7.1 — MICE-pooled (4 Variations): OC pooled across draws via Rubin's Rules. Holes computed from imputation 1 only. Utilizes the global Tri-Language Grand Mean calculation to render four distinct bivariate maps (`GrandMean`, `Python`, `R`, `Julia`). Tertile breaks applied independently to each dimension.

7.2 — Observed-only: Both OC and Holes restricted to the same Phase 2 observed-acreage subset (`acreage_source != "MICE_Target"`), keeping the two bivariate dimensions consistent. Tertile breaks recomputed independently on the observed distribution.

A shared `build_bivariate_map()` function handles `bi_class`, map rendering, and `cowplot` assembly for both outputs.

Key inputs: `R_Imputed_Dataset_{1..100}.csv`, `R_Phase1_Baseline_Golf_Valuation.csv`, `R_Phase2_Acreage_Matched_v2.csv`
R packages: `tidyverse`, `sf`, `tigris`, `biscale`, `cowplot`, `this.path`

6.3.8 Script 8 — `8_LaTeX_Tables.R`

Outputs: `output/8.1_Table1_Acreage.tex`, `output/8.2_Table2_Regression.tex`, `output/8.3_Table3_Hawaii_Geo.tex`

Generates three standalone `booktabs`-styled LaTeX table files for direct `\input{}` inclusion in the thesis. All tables include `\caption{}` and `\label{}`. A `latex_escape()` helper escapes `%`, `$`, `_`, `&`, `#`, `^`, `~`, and backslashes before passing data to `knitr::kable(..., escape = FALSE)`.

File	Source	Content
`8.1_Table1_Acreage.tex`	`National_Acreage_Summary.csv`	Urban / Rural / National pooled acreage with 95% CI; numbers formatted with thousand-separator commas
`8.2_Table2_Regression.tex`	`R_Regression_Results.csv`	Coefficients, SEs, t-stats, adjusted df, p-values, significance stars, and FMI; p-values below 0.001 shown as `< 0.001`; `threeparttable` footnote
`8.3_Table3_Hawaii_Geo.tex`	`Phase5_Geographic_Breakdown.csv`	Zone, district name, parcel count, share (%); percentages rounded to 1 decimal place

LaTeX preamble requirements: `booktabs`, `threeparttable`, `float`
R packages: `tidyverse`, `knitr`, `kableExtra`, `this.path`

6.3.9 Script 9 — `9_Oahu_Opportunity_Cost_Map.R`

Outputs: `output/9.1_Oahu_Opportunity_Cost_Map.png` (12 × 10 in), `output/9.2_Oahu_Opportunity_Cost_` (12 × 10 in)

High-resolution micro-map of Oahu coloring individual OSM golf course polygons by their Opportunity Cost, mirroring the plasma-scale aesthetic of the national macro maps at the course level.

Data pipeline (both variants): OC coordinates are converted to sf points (EPSG 4326 → 32604), then each polygon is matched to its nearest point via `st_nearest_feature()`; matches exceeding 500 m are discarded. Island base derived from `st_union()` dissolve of

Honolulu parcels (consistent with Script 3). A shared `join_oc_to_polygons()` helper and `build_oahu_oc_map()` function serve both variants.

9.1 — MICE-pooled: Imputed datasets filtered to Oahu bounding box (Lat: 21.2–21.9, Lon: −158.5–−157.6). `Total_Opportunity_Cost = final_acreage × Baseline_Value_Per_Acre` averaged across M = 100 draws (Rubin's Rules $\bar{q}$).

9.2 — Observed-only: Phase 2 data filtered to Oahu bounding box and `acreage_source != "MICE_Target"`. Direct OC sum per course; no pooling. Polygons with no nearby observed-acreage coordinate within 500 m rendered in neutral gray.

Fill: `scale_fill_viridis_c(option = "plasma")` with a custom `label_oc()` labeler (auto-scales `$M` / `$B`). North arrow top-right; scale bar bottom-right via `ggspatial`. `theme_void()`.

Key inputs: `Target_Golf_Polygons.gpkg`, `R_Imputed_Dataset_{1..100}.csv`, `Honolulu_Parcels_Reproje`
`R_Phase2_Acreage_Matched_v2.csv`
R packages: `tidyverse`, `sf`, `scales`, `ggspatial`, `this.path`

6.3.10 Script 9b — `9b_Oahu_OC_Map_Rural_USDA_Sensitivity.R`

Output: `output/9b.141_Oahu_OC_Map_Rural_USDA_Sensitivity_GrandMean.png` (defense deliverable, 12 × 10 in)

Defense-only sensitivity variant of the Oahu Grand Mean opportunity cost map. Reclassifies courses in rural Honolulu County Development Plan zones from the FHFA residential proxy to the USDA agricultural proxy. Addresses the FHFA-aggregation caveat in thesis §5.4: Honolulu County's countywide FHFA index does not distinguish rural submarkets from the urban core.

Reclassification logic: `ZONMAP_NO` from `Zoning_Map_Boundary.geojson` (City & County of Honolulu Development Plan boundary layer, 34 polygons): - Codes 1–14, 21–24 (urban/suburban core; Windward Oahu: Kualoa, Kaneohe, Kailua, Waimanalo): FHFA ($4,952,600/ac, FIPS 15003, 2022) - Codes 15–20 (Lualualei/Makaha, Makua/Kaena, Mokuleia/Wailua/Haleiwa, Kawailoa/Waialee, Kahuku/Laie, Hauula/Punaluu/Kaaawa): USDA ($29,887/ac, read dynamically from `2022 - USDA County Data - Ag Use.csv`) - Code 0: no golf courses present

FHFA normalization step: ALL Oahu `Baseline_Value_Per_Acre` values are overwritten with the FHFA value before any USDA override. This corrects Hawaii Kai and Mid-Pacific (FIPS-NA courses whose BVPA was MICE-imputed in Py/Jl rather than assigned the Phase 1 FHFA value).

Spatial pipeline: `st_centroid` of each OSM golf polygon → `st_join(st_within)` against Development Plan polygons → `st_nearest_feature` fallback for unmatched centroids (500m cap). Grand Mean: Rubin's Rules independently per language (M=100 each), arithmetic mean

of three pooled estimates. Map format matches Script 9's `9.141` output (same plasma color scale, polygon-to-point join, caption format).

Integrated into `Phase_6.R` as `run_9b_Oahu_OC_Rural_USDA_Sensitivity()`, output routed to `output/Final_Thesis_Figures/`.

Key inputs: `Target_Golf_Polygons.gpkg`, `Honolulu_Parcels_Reprojected.gpkg`, `R_Phase1_Baseline_Golf_Valuation.csv`, `Zoning_Map_Boundary.geojson`, `R/Py/Jl_Imputed_Dataset_{1.` `2022 - USDA County Data - Ag Use.csv`
R packages: `tidyverse`, `sf`, `scales`, `ggspatial`, `this.path`

6.3.11 Script 15 — `15_Residual_Map.R`

Outputs: `output/15.141_Log_Residual_Map_GrandMean.png` (14 × 9 in), `output/15.241_Dollar_Residual_` (14 × 9 in)

Two residual choropleth maps at the county level. Residuals measure the gap between each course's observed opportunity cost and the OLS-predicted value from the Phase 4 regression (`log(OC) = + ·Holes + ·I(Urban)`). Tri-language Grand Mean: Rubin's Rules independently per language (M=100 each), arithmetic mean of three pooled county-level estimates.

15.141 — Log-residual map: `log(acreage × BVPA) - predicted_log`, aggregated to county mean. Range: [−2.809, 4.437]. Zero = perfect model fit; positive = observed OC exceeds prediction (undervalued by the model); negative = prediction exceeds observed OC.

15.241 — Dollar-residual map: `(acreage × BVPA) - exp(predicted_log)`, both terms in dollars. Range: [$−1.30B, $45.86B]. Counties with residuals: 2,874 of 3,144 (288 counties have no course data, rendered gray).

Data safety filters: - `acreage > 1`: guards against `log(0)` / `log(1e-15)` producing −Inf or NaN from near-zero MICE-imputed acreage - `between(Holes, 9, 72)`: guards against the 252-hole Phase 1 aggregate record causing `exp(b_holes × 252)` `$3.7T` explosion in the dollar-residual map - FIPS-safe cross-language join: `select(-any_of(c("FIPS", "County_Name", "State_Abbr", "Tigris_State_Abbr")))` before `left_join(county_lookup, ...)` — eliminates the R FIPS-not-found error and the Julia `State_Abbr` column-suffix conflict

These maps are spatial diagnostics; the thesis prose does not reference any `15.x` figure.

Key inputs: `R/Py/Jl_Imputed_Dataset_{1..100}.csv`, `R_Phase1_Baseline_Golf_Valuation.csv`
R packages: `tidyverse`, `sf`, `tigris`, `scales`, `this.path`

6.4 R Output File Index

```
Bulk/R/output/
  1.141_National_Opportunity_Cost_Map_[Lang].png       (14 × 9 in, 300 DPI)
  1.101_National_Opportunity_Cost_Map_ObservedOnly.png (14 × 9 in, 300 DPI)
  2.141_County_Opportunity_Cost_Map_[Lang].png         (14 × 9 in, 300 DPI)
  2.101_County_Opportunity_Cost_Map_ObservedOnly.png   (14 × 9 in, 300 DPI)
  3.101_Oahu_TMK_Concentration_Map.png                 (12 × 10 in, 300 DPI)
  4.101_Oahu_Golf_Zoning_Map.png                       (12 × 10 in, 300 DPI)
  7.141_Bivariate_Cost_vs_Density_Map_[Lang].png       (14 × 9 in, 300 DPI)
  7.101_Bivariate_Cost_vs_Density_Map_ObservedOnly.png (14 × 9 in, 300 DPI)
  8.141_Table1_Acreage.tex
  8.241_Table2_Regression.tex
  8.301_Table3_Hawaii_Geo.tex
  9.131_Oahu_Opportunity_Cost_Map_R.png                          (12 × 10 in, 300 DPI)
  9.101_Oahu_Opportunity_Cost_Map_ObservedOnly.png               (12 × 10 in, 300 DPI)
  9b.141_Oahu_OC_Map_Rural_USDA_Sensitivity_GrandMean.png        (12 × 10 in, 300 DPI)
  15.141_Log_Residual_Map_GrandMean.png                          (14 × 9 in, 300 DPI)
  15.241_Dollar_Residual_Map_GrandMean.png                       (14 × 9 in, 300 DPI)
```

6.5 Julia Bulk Scripts

Julia scripts live under `Bulk/Julia/` and cover all non-spatial outputs: statistical plots and LaTeX table generation. All scripts use `@__DIR__` for path resolution, read only `Jl_`-prefixed input files, and save PNGs at `px_per_unit = 3` (300 DPI equivalent). Scripts use `CairoMakie.jl` for all plotting.

Spatial map scripts are **not translated to Julia** — those outputs are consolidated into `Phase_6.R`. Script 8 (LaTeX tables) and Advanced Plots (10-14) are fully executed by `Phase_6.jl`.

6.5.1 Script 5 — `5_Econometric_Plots.jl`

Outputs: `output/Final_Thesis_Figures/5.141_Forest_Plot_Combined.png`, `output/Final_Thesis_Figu`

5.1 — Forest Plot: Combines the pooled regression coefficients for all three languages. Points and 95% CIs (`Coef ± 1.96 × SE`) are plotted side-by-side per variable using a Y-axis dodge

(±0.2) to perfectly display Python (Green), R (Blue), and Julia (Purple) estimates without overlap.

5.2 — MICE Density Plot: Visualizes the true density variance of the full MICE pipeline by sequentially overlaying all 300 imputed datasets. The script features a highly optimized memory-management loop that dynamically matches missing coordinates, overlays a single 0.02 opacity density trace (color-coded by language), and immediately clears the DataFrame from RAM. The resulting 300-draw "cloud" is plotted behind the black observed acreage curve.

Key inputs: Phase 4 Regression CSVs, Phase 2 Matched Acreage CSVs, Phase 3 Imputed Datasets (300 files)
Julia packages: CSV, DataFrames, CairoMakie, Printf

6.5.2 Script 6 — 6_Advanced_Econometric_Plots.jl

Outputs: output/6.1_Marginal_Effects_Dollar_Value.png, output/6.2_MICE_Raincloud_Diagnostic.pr

6.1 — Marginal Effects Plot: Computes predicted dollar opportunity costs for Rural and Urban courses at median holes via exp(log_hat) × bvpa / 1e6. Delta method CIs: se_pred_rural = √(se_b0² + (med_holes × se_holes)²); adds se_urban² for Urban. Rendered with vertical errorbars!, a double-scatter (white backing + colored fill) to replicate R's outlined-circle point style, and text! value labels above the CI upper bound.

6.2 — Raincloud Diagnostic: Raincloud (Observed + 100 MICE draws) built from three CairoMakie primitives per group — violin!(side = :right) (half-violin density slab), boxplot! (narrow centered box, outliercolor = :transparent), and jittered scatter!. Replicates ggdist::stat_halfeye. Random.seed!(42) applied before jitter. Log -transformed acreage on the y-axis with manual tick labels.

Key inputs: Jl_Regression_Results.csv, Jl_Phase2_Acreage_Matched.csv, Jl_Imputed_Dataset_{1..10
Julia packages: CSV, DataFrames, CairoMakie, Statistics, Printf, Random

---

6.5.3 Script 8 — 8_LaTeX_Tables.jl

Outputs: output/8.1_Table1_Acreage.tex, output/8.2_Table2_Regression.tex, output/8.3_Table3_Hawaii_Geo.tex

Generates the same three booktabs-styled LaTeX tables as 8_LaTeX_Tables.R. Because Julia has no kableExtra equivalent, LaTeX markup is constructed directly by string-building into

a `Vector{String}` and written via `open/println`. Structural output is equivalent: `[H]` placement, `\toprule/\midrule/\bottomrule` rules, `threeparttable` footnote for Table 2.

A `latex_escape()` helper replicates the R function (backslash processed first). `fmt_comma()` replicates `format(..., big.mark = ",")` by chunking the integer part into groups of three. `fmt_pval()` replicates R's `ifelse(p < 0.001, "$<$ 0.001", sprintf("%.3f", p))`. Parameter name `"county_type: Urban"` used in `PARAM_LABELS` to match the Julia regression CSV.

File	Source	Content
`8.1_Table1_Acreage.tex`	`Jl_National_Acreage_Summary.csv`	Urban / Rural / National pooled acreage with 95% CI; comma-formatted numbers
`8.2_Table2_Regression.tex`	`Jl_Regression_Results.csv`	Coefficients, SEs, t-stats, adjusted df, p-values, significance stars, FMI; `threeparttable` footnote
`8.3_Table3_Hawaii_Geo.tex`	`Jl_Phase5_Geographic_Breakdown.csv`	Zoning district, parcel count, share (%)

LaTeX preamble requirements: `booktabs`, `threeparttable`, `float`
Julia packages: `CSV`, `DataFrames`, `Printf`

6.5.4 Scripts 10–14 — Advanced Statistical Plots

Scope: The advanced statistical scripts (`10_Hawaii_Gap_Dumbbell.jl`, `11_Lorenz_Curve.jl`, `12_Zoning_Waffle_Chart.jl`, `13_Counterfactual_Area.jl`, `14_Urban_Rural_Scatter.jl`) use CairoMakie to render multi-dimensional visual comparisons of the economic implications.
Data Routing: Rather than overlapping multi-language series, these final figures represent the absolute definitive thesis outcomes by actively routing and plotting the global **Tri-Language Grand Mean** (`$0.938T`) or the static observed baseline.

6.6 Output File Index

Both `Phase_6.R` and `Phase_6.jl` render their final consolidated outputs into the unified `Phase 6 Visualization/output/` directory, separated by the thesis publication state.

```
output/
  Final_Thesis_Figures/
      1.101_National_Opportunity_Cost_Map_ObservedOnly.png        [Phase_6.R Script 1]
      1.141_National_Opportunity_Cost_Map_GrandMean.png           [Phase_6.R Script 1]
      2.101_County_Opportunity_Cost_Map_ObservedOnly.png          [Phase_6.R Script 2]
      2.141_County_Opportunity_Cost_Map_GrandMean.png             [Phase_6.R Script 2]
      3.101_Oahu_TMK_Concentration_Map.png                        [Phase_6.R Script 3]
      4.101_Oahu_Golf_Zoning_Map.png                              [Phase_6.R Script 4]
      5.141_Forest_Plot_Combined.png                              [Phase_6.jl Mod_5]
      5.241_MICE_Density_Combined_n020.png                        [Phase_6.jl Mod_5]
      5.242_MICE_Density_Combined_n040.png                        [Phase_6.jl Mod_5]
      5.243_MICE_Density_Combined_n060.png                        [Phase_6.jl Mod_5]
      5.244_MICE_Density_Combined_n080.png                        [Phase_6.jl Mod_5]
      5.245_MICE_Density_Combined_n100.png                        [Phase_6.jl Mod_5]
      6.141_Marginal_Effects_Dollar_Value_Combined.png            [Phase_6.jl Mod_6]
      6.241_MICE_Raincloud_Diagnostic_Combined.png                [Phase_6.jl Mod_6]
      7.101_Bivariate_Cost_vs_Density_Map_ObservedOnly.png        [Phase_6.R Script 7]
      7.141_Bivariate_Cost_vs_Density_Map_GrandMean.png           [Phase_6.R Script 7]
      8.141_Table1_Acreage.tex                                    [Phase_6.R Script 8]
      8.241_Table2_Regression.tex                                 [Phase_6.R Script 8]
      8.301_Table3_Hawaii_Geo.tex                                 [Phase_6.R Script 8]
      9.101_Oahu_Opportunity_Cost_Map_ObservedOnly.png                [Phase_6.R Script 9]
      9.141_Oahu_Opportunity_Cost_Map_GrandMean.png                   [Phase_6.R Script 9]
      9b.141_Oahu_OC_Map_Rural_USDA_Sensitivity_GrandMean.png         [Phase_6.R Script 9b]
      10.141_Hawaii_Gap_Dumbbell_TriLanguage.png                      [Phase_6.jl Mod_10]
      11.141_Lorenz_Curve_TriLanguage.png                         [Phase_6.jl Mod_11]
      12.141_Zoning_Waffle_Chart_TriLanguage.png                  [Phase_6.jl Mod_12]
      13.141_Counterfactual_Area_TriLanguage.png                  [Phase_6.jl Mod_13]
      14.141_Urban_Rural_Scatter_TriLanguage.png                  [Phase_6.jl Mod_14]
      15.141_Log_Residual_Map_GrandMean.png                       [Phase_6.R Script 15]
      15.241_Dollar_Residual_Map_GrandMean.png                    [Phase_6.R Script 15]

  QA_Verification/
      1.111_National_Opportunity_Cost_Map_Julia.png               [Phase_6.R Script 1]
      1.121_National_Opportunity_Cost_Map_Python.png              [Phase_6.R Script 1]
      1.131_National_Opportunity_Cost_Map_R.png                   [Phase_6.R Script 1]
      2.111_County_Opportunity_Cost_Map_Julia.png                 [Phase_6.R Script 2]
      2.121_County_Opportunity_Cost_Map_Python.png                [Phase_6.R Script 2]
      2.131_County_Opportunity_Cost_Map_R.png                     [Phase_6.R Script 2]
      5.211_MICE_Density_Jl_n020.png  …  5.215_MICE_Density_Jl_n100.png  (5 files)  [Phase_6
      5.221_MICE_Density_Py_n020.png  …  5.225_MICE_Density_Py_n100.png  (5 files)  [Phase_6
      5.231_MICE_Density_R_n020.png   …  5.235_MICE_Density_R_n100.png   (5 files)  [Phase_6
```

```
7.111_Bivariate_Cost_vs_Density_Map_Julia.png          [Phase_6.R Script 7]
7.121_Bivariate_Cost_vs_Density_Map_Python.png         [Phase_6.R Script 7]
7.131_Bivariate_Cost_vs_Density_Map_R.png              [Phase_6.R Script 7]
```

Note (6C audit): Script 9 (Oahu OC Map) produces only GrandMean + ObservedOnly in `Final_Thesis_Figures/` — no per-language QA maps. Script 15 (Residual Maps) produces GrandMean only — no ObservedOnly variant by design (residuals require a fitted model).

7 Phase 7 Summary: Documentation, Discussion & Write-Up

7.1 Thesis Coverage Summary — Phases 1 through 6

This document synthesizes the goal, purpose, intent, and accomplishments of each of the six computational phases of the thesis. The overarching research question is: **What is the aggregate opportunity cost of U.S. golf course land, and what does it imply about how that land is valued and used?** Each phase addresses a distinct layer of that question, from raw data parsing through visualization.

7.2 Phase 1 — Spatial Parsing & Economic Baseline Valuation

7.2.1 Goal and Purpose

Phase 1 builds the foundational dataset for the entire thesis. The goal is to transform a raw GPS-coordinate dataset of golf courses into a structured, spatially-validated, economically-anchored baseline. Every course must receive a county FIPS assignment and an estimated land value per acre before any further analysis can proceed.

7.2.2 Intent

The core methodological intent is to estimate what each golf course's land would be worth in its next-highest-value use — the Highest and Best Use (HBU) framework. Rather than using a single appraisal methodology, Phase 1 introduces a **dual-proxy valuation**: urban counties (RUCC codes 1–3) receive the FHFA Residential Land Price index as the opportunity cost benchmark (reflecting the residential development market), while rural counties (RUCC codes 4–9) receive the USDA Agricultural Land Value (reflecting the agricultural land market). This classification is derived from 2023 USDA Rural-Urban Continuum Codes applied at the county level.

The pipeline is implemented independently in **Python**, **R**, and **Julia** as a robustness strategy: three entirely separate computational stacks reading the same source data must converge on statistically equivalent outputs. Any divergence signals a pipeline defect rather than a substantive finding.

7.2.3 Accomplishments

- Parsed 16,292–16,297 golf courses from raw GPS coordinates, extracting `Course_Type` and `Holes` via regex.
- Performed a spatial point-in-polygon join against 2022 US Census county boundaries to assign 5-digit FIPS codes to every course. Identified and fixed a FIPS **zero-padding bug** across all three language implementations (integer coercion silently dropped leading zeros, causing hundreds of USDA/FHFA join failures before the fix).
- Merged USDA Agricultural Land Values and FHFA Residential Land Prices onto each course by FIPS, then applied the RUCC Urban/Rural classification to select the correct proxy.
- Produced a `Baseline_Value_Per_Acre` for 15,197–15,198 courses. The remaining **~1,094–1,095 courses** lacked a baseline value (due to missing county data, USDA suppression, or coordinates outside CONUS) and were flagged as MICE imputation targets for Phase 3.
- Verified cross-language statistical parity: all three pipeline means converge within $5 of each other (~$413,700/acre) despite minor row-count differences at the margins attributable to `geopandas` spatial deduplication behavior in Python.
- Standardized all outputs with language-specific prefixes (`Py_`, `R_`, `Jl_`) to prevent filename collisions across phases.

Output files: `{Py|R|Jl}_Phase1_Baseline_Golf_Valuation.csv`

7.3 Phase 2 — OSM Polygon Extraction & True Acreage Matching

7.3.1 Goal and Purpose

Phase 1 established *what* each course is worth per acre. Phase 2 establishes *how many acres* each course occupies. The goal is to replace the implicit assumption of a fixed or average acreage with the true measured polygon area derived from OpenStreetMap (OSM) golf course boundary data.

7.3.2 Intent

True acreage is the physical multiplier for the opportunity cost calculation: `Total_Opportunity_Cost = osm_acreage × Baseline_Value_Per_Acre`. Without it, the aggregate national figure is meaningless. OSM is the most comprehensive freely available source of golf course boundary polygons, but its coverage is imperfect — not every course in the Phase 1 dataset has a corresponding polygon. Phase 2's intent is to maximize match coverage using a two-tier spatial

strategy and to clearly identify the residual unmatched cases for MICE imputation in Phase 3.

The phase also introduces the concept of **acreage sourcing**: every course in the output is tagged with `acreage_source` ("OSM", "Tigris", or "MICE_Target") so that downstream analyses can distinguish directly measured values from imputed ones.

7.3.3 Accomplishments

- Extracted golf course polygons from the 11 GB US OpenStreetMap PBF file using `pyosmium` streaming. After filtering size outliers (< 5 acres = fragments; > 1,500 acres = mega-resort blobs), retained **15,166 valid polygons** with a median area of 127.8 acres.
- All area calculations were performed in EPSG:5070 (NAD83/Conus Albers), a planar equal-area projection, rather than in geographic degrees, ensuring geometrically correct acreage values.
- Matched Phase 1 GPS point coordinates to OSM polygons via a **two-tier join**: (1) direct spatial intersect, then (2) nearest-neighbor fallback within 500 m for courses whose listed coordinate falls outside the polygon boundary (e.g., clubhouse or parking lot coordinates).
- Achieved a **71.2% match rate** (11,605 of 16,292 courses), leaving **4,687 courses** (28.8%) without an OSM acreage value. These are the primary MICE target in Phase 3.
- R additionally applied a Tigris landmarks fallback tier, modestly recovering additional courses, creating a documented asymmetry between R's three-value acreage source schema ("OSM" | "Tigris" | "MICE_Target") and Python/Julia's two-value schema ("OSM" | "MICE_Target"). This asymmetry propagates consistently through all downstream phases.
- Standardized outputs with language-specific prefixes and added `acreage_source` column to all three output CSVs.

Output files: `{Py|R|Jl}_Phase2_Acreage_Matched.csv`, `{Py|R|Jl}_Phase2_OSM_Golf_Polygons.gpkg`

7.4 Phase 3 — MICE Imputation & Rubin's Rules Aggregate Valuation

7.4.1 Goal and Purpose

Phase 3 addresses the missingness problem inherited from Phases 1 and 2: approximately 28.8% of courses lack `osm_acreage` and 6.7% lack `Baseline_Value_Per_Acre`. The goal is to produce statistically principled complete datasets through Multiple Imputation by Chained Equations (MICE), then aggregate the resulting national opportunity cost estimates using Rubin's Rules for valid inference under multiple imputation.

7.4.2 Intent

A complete-case analysis that simply drops courses with missing values would be both wasteful and biased: if courses with missing data are systematically different from those with complete data (e.g., smaller or lower-value), the aggregate estimate would be distorted. MICE avoids this by generating M = 100 plausible complete datasets per language, each consistent with the observed data distribution, and then pooling the per-dataset estimates using Rubin's Rules to propagate imputation uncertainty into the final confidence intervals.

The 28.8% missing acreage rate is high enough that imputation uncertainty is a non-trivial contributor to the total variance in the national estimate — which is precisely why M = 100 (rather than the historically suggested M = 3–10) is used: modern standards (Graham et al., 2007; von Hippel, 2020) require larger M to stabilize standard errors at this missingness rate.

A tree-based (Random Forest / LightGBM) MICE algorithm is used because land values and course sizes have non-linear, geography-dependent relationships that linear models would inadequately capture, and because tree-based algorithms predict strictly within the observed range (preventing impossible negative acreages or land values).

7.4.3 Accomplishments

- Ran MICE independently in three languages with three different algorithmic backends:
  - **Python:** `miceforest` v6.0.5 with LightGBM gradient-boosted imputation
  - **R:** `mice` package with Random Forest (`method = "rf"`) via `futuremice()` parallel execution
  - **Julia:** `Mice.jl` with its native Random Forest default
- Produced **300 complete imputed datasets** (100 per language), each prefixed to its generating pipeline, with all predictor variables (`Holes`, `Course_Type/Ownership_Type`, `county_type`, `Longitude`, `Latitude`) and both imputation targets (`osm_acreage`, `Baseline_Value_Per_Acre`) included.
- Applied Rubin's Rules independently within each language group (M = 100) to produce pooled national aggregate opportunity cost estimates with 95% confidence intervals:

Language	Pooled $\bar{Q}$	95% CI
Python	\$943.025 B	\$936.3 B — \$949.7 B
R	\$936.046 B	\$926.4 B — \$945.7 B
Julia	\$951.389 B	\$943.8 B — \$958.9 B

- All three 95% CIs overlap substantially, confirming cross-language statistical agreement. The ~\$15B spread (1.6% of the ~\$940B base) is attributable to different Random Forest RNG seeds and internal MICE implementations — not a methodological discrepancy.

- Verified that between-imputation variance (V_B) dominates within-imputation variance (V_W) by 2–3 orders of magnitude in all three languages — the expected behavior for a well-specified MICE model.
- A complete-case analysis using only courses with directly observed data ($943B) falls within all three MICE confidence intervals, confirming result robustness despite imputing 68.4% of the sample.

Grand Mean across three languages: ~$943 billion. Range: $926B – $959B.

Output files: `{Py|R|Jl}_Imputed_Dataset_{1..100}.csv`, `{Py|R|Jl}_Rubins_Rules_Summary.csv`, `{Py|R|Jl}_National_Acreage_Summary.csv`

7.5 Phase 4 — Econometric Modeling

7.5.1 Goal and Purpose

Phase 4 moves from aggregate valuation to structural estimation. The goal is to fit a log-linear OLS regression model that isolates the relationship between observable course characteristics (number of holes, urban/rural location) and log opportunity cost, using all 100 MICE-imputed datasets per language and pooling the resulting coefficient estimates via Rubin's Rules.

7.5.2 Intent

The aggregate figure from Phase 3 tells us the total value at stake. The regression in Phase 4 tells us *why* that value is distributed the way it is across the country. Two structural forces are hypothesized: (1) **course size** (`Holes`) as a proxy for physical capacity and land area, and (2) **geographic context** (`county_type`: Urban vs. Rural) as the primary driver of land market value. The log-linear specification `log(Opportunity_Cost) ~ Holes + county_type` captures these relationships while handling the extreme right-skew of the dollar-valued outcome.

HC1 heteroskedasticity-robust standard errors are applied throughout because cross-sectional land-value data almost certainly violates the OLS homoskedasticity assumption (variance of residuals is plausibly larger in high-value urban areas). Pooling coefficient estimates across M = 100 imputations per language via Rubin's Rules propagates imputation uncertainty into the reported standard errors and confidence intervals.

7.5.3 Accomplishments

- Implemented the complete OLS + HC1 + Rubin's Rules pipeline in all three languages:
  - **Python:** `statsmodels.formula.api.ols` with `cov_type="HC1"`
  - **R:** `lm()` with `sandwich::vcovHC(type="HC1")`
  - **Julia:** Manual HC1 sandwich estimator computed directly from `modelmatrix()` and `residuals()`
- Pooled 100 per-dataset coefficient estimates per language via Rubin's Rules (Barnard & Rubin, 1999 adjusted degrees of freedom). Final pooled coefficients:

Parameter	R	Julia	Python
Intercept	12.229	12.247	12.282
Holes	0.0525	0.0476	0.0474
Urban County	4.001	4.158	4.172
Mean R^2	0.699	0.730	~0.770

- **Key finding — Urban land premium:** The `county_type: Urban` coefficient of ~4.0–4.2 log-units implies that urban golf course land is worth approximately `exp(4.1)` `60×` more per acre than equivalent rural land. This is by far the largest effect in the model and is estimated with t-statistics exceeding 150 in every implementation.
- **Key finding — Course size effect:** Each additional hole is associated with a 4.7–5.3% increase in log opportunity cost across all three languages, consistent with larger courses concentrating in higher-value geographic areas.
- **Cross-language consistency confirmed:** All three independent MICE/OLS stacks converge on qualitatively identical conclusions. Minor coefficient differences (~10% spread on the Holes coefficient) are traced to R's use of `final_acreage` (OSM + Tigris) vs. Python/Julia's `osm_acreage` (OSM-only) — a documented Phase 2 asymmetry.
- Model explains 69–77% of variance in log opportunity cost across all languages (strong for a two-predictor cross-sectional model), driven primarily by the Urban/Rural distinction.

Output files: `{Py|R|Jl}_Regression_Results.csv`, `{Py|R|Jl}_model_results.{pkl|rds|jls}`

7.6 Phase 5 — Hawaii Micro-Case Study

7.6.1 Goal and Purpose

Phase 5 grounds the national model in a specific, verifiable empirical context. The goal is to validate whether the thesis's HBU-based opportunity cost estimates align with real-world

property tax assessments for Hawaii golf courses, and to characterize the regulatory landscape (zoning, geographic distribution) that would govern any hypothetical land conversion.

7.6.2 Intent

A purely national aggregate estimate risks being dismissed as a theoretical exercise disconnected from actual land markets. Hawaii is selected as the validation site for several reasons: (1) it has high-value, well-documented golf courses in both urban (Honolulu) and rural (Big Island, Kauai) settings; (2) official Honolulu County parcel-level cadastral and zoning data are publicly available at TMK (Tax Map Key) resolution; and (3) Hawaii's geography compresses a wide range of opportunity cost scenarios — from resort-zone courses in Honolulu ($866M average OC per course) to rural Big Island courses ($1.7M average) — into a small, tractable area.

The micro-case study also interrogates the HBU assumption itself: if most golf land sits in Preservation or Federal Military zones where residential redevelopment is legally prohibited, the theoretical opportunity cost is economically meaningful only for the subset of land in zones where conversion is actually permitted.

7.6.3 Accomplishments

- **Phase 5a (Pilot):** Manually compared HBU estimates against official tax assessments for 6 high-profile Hawaii courses. Average model-to-assessed ratio: **1.33x** (model estimates are 32.6% higher on average). The gap is largest for Big Island rural courses (1.49x–1.69x) and narrowest for Honolulu urban courses (1.16x–1.23x), consistent with the hypothesis that the urban FHFA proxy better tracks market values than the rural USDA proxy.
- **Phase 5b (Full Pipeline — Honolulu County):**
  - Built a 6-step automated spatial pipeline extracting OSM polygons for Oahu, intersecting them against the Honolulu County cadastral database, performing Rubin's Rules economic validation ($M = 100$, $\bar{q} =$ **$25.40B**, 95% CI: $22.66B–$28.14B), and analyzing parcel-level zoning composition.
  - Identified **1,073 unique TMKs** intersecting Oahu golf polygons across 8,564.23 acres of OSM-derived legal footprint.
  - **Geographic concentration (Step 5):** 63.2% of all golf parcels fall in the Ewa district (Zone 9, Kapolei/Pearl City), reflecting Oahu's post-statehood suburban golf development corridor rather than the urban core.
  - **Zoning analysis (Step 6):** 81.7% of all Oahu golf land sits within Preservation (P-1, P-2) or Federal/Military (F-1) zones — areas where residential redevelopment faces the highest regulatory barriers. Only 2.2% of golf land is in Resort-zoned areas, though golf occupies a striking **25.4%** of all Resort-zoned land on the island.

– These findings materially qualify the HBU framework: the $25.4B Oahu aggregate opportunity cost is theoretically correct under unrestricted redevelopment but practically constrained by zoning for over 80% of the acreage.

Output files: `{Py|R|Jl}_Phase5_Oahu_Comparison.csv`, `{Py|R|Jl}_Phase5_Geographic_Breakdown.csv`, `{Py|R|Jl}_Phase5_Step6_Zoning_Percentages.csv`, `{Py|R|Jl}_Phase5_Step6_Zone_Golf_Penetration.cs`

7.7 Phase 6 — Visualization

7.7.1 Goal and Purpose

Phase 6 converts all upstream computational outputs into the publication-ready figures, maps, and LaTeX tables that will appear in the thesis document. The goal is a complete, reproducible visualization suite covering every major finding from Phases 1–5.

7.7.2 Intent

Economic research is persuasive only when findings are presented clearly. Phase 6's intent is threefold: (1) **spatial legibility** — national and regional choropleth maps that allow readers to immediately identify geographic concentration of golf course opportunity cost; (2) **statistical transparency** — forest plots, density overlays, and diagnostic charts that expose the full MICE pipeline and cross-language coefficient comparison rather than hiding uncertainty behind a single point estimate; and (3) **structural insight** — advanced figures (Lorenz curves, counterfactual area charts, zoning waffle charts) that contextualize the distributional and regulatory dimensions of the finding.

The language assignment strategy explicitly optimizes for output quality: R handles all spatial/cartographic outputs where its `sf + ggplot2 + tigris` stack excels; Julia handles all statistical chart outputs where CairoMakie produces cleaner publication-quality figures than Python's matplotlib or R's base graphics for non-spatial work.

7.7.3 Accomplishments

- **Master script architecture:** Two standalone master scripts — `Phase_6.R` (~2,280 lines) and `Phase_6.jl` (~1,843 lines) — independently reproduce the full visualization pipeline without calling bulk scripts. All outputs route automatically to either `output/Final_Thesis_Figures/` (Grand Mean and ObservedOnly variants, for direct thesis inclusion) or `output/QA_Verification/` (per-language Julia/Python/R variants, for internal cross-language QA).

- **Tri-Language Grand Mean:** Both master scripts implement `compute_grand_means()` / equivalent logic that applies Rubin's Rules independently within each language group (M = 100 each) and then computes the Grand Mean as the arithmetic mean of the three independently pooled estimates. This produces the definitive \$0.944T national aggregate, with the observed-only baseline at \$0.788T (82.5% of the MICE-pooled figure).
- **National and county maps (Scripts 1–2, R):** State-level and county-level choropleth maps of total opportunity cost, each rendered in four variants (Grand Mean, Julia, Python, R) plus an observed-only baseline. Alaska and Hawaii repositioned as insets. Plasma colormap on linear (state) and log (county) scales.
- **Oahu micro-maps (Scripts 3, 4, 9, R):** High-resolution maps of all 1,072 golf course TMK parcels on Oahu, colored by geographic district (Script 3), dominant zoning class (Script 4), and per-polygon opportunity cost (Script 9). Script 9 applies Rubin's Rules pooling over M = 100 R draws, yielding a pooled Oahu OC of \$31.197B across 37 courses.
- **Bivariate map (Script 7, R):** County-level bivariate choropleth jointly displaying opportunity cost and golf course hole density via a 3×3 biscale classification, highlighting counties with both high land value and high golf intensity.
- **LaTeX tables (Script 8, R and Julia):** Three `booktabs`-styled tables for direct `\input{}` inclusion — national acreage summary with CIs, OLS regression coefficients with robust SEs and FMI, and Oahu geographic breakdown by TMK district.
- **Forest plot (Script 5, Julia):** Tri-language clustered forest plot displaying all three language coefficient estimates (Intercept, Holes, Urban County) with 95% CIs, color-coded Green (Python), Blue (R), Purple (Julia), confirming cross-language agreement.
- **MICE density overlays (Script 5, Julia):** All 300 imputed datasets overlaid as low-opacity density traces (color-coded by language) behind the black observed-acreage curve, with five checkpoint snapshots at n = 20, 40, 60, 80, 100 per language.
- **Marginal effects and raincloud diagnostics (Script 6, Julia):** Dollar-scale marginal effects plot translating log-coefficient to predicted Rural vs. Urban opportunity costs with delta-method CIs; raincloud diagnostic replicating `ggdist::stat_halfeye` from CairoMakie primitives.
- **Advanced structural figures (Scripts 10–14, Julia):** Dumbbell gap chart (Hawaii vs. national), Lorenz curve (opportunity cost concentration), zoning waffle chart, counterfactual area chart, and Urban/Rural scatter — all tri-language Grand Mean figures for the definitive thesis presentation.
- **Residual maps (Script 15, R):** County-level log-residual and dollar-residual choropleths from the Phase 4 Grand Mean OLS fit, identifying geographic areas where the model systematically over- or under-predicts opportunity cost.

- **1.234 naming convention:** All 40+ output files follow the four-digit structural naming format (`MainScript.SubCategoryLanguageIDSubCount`), enabling exact data lineage tracing from filename alone.

Top-line findings confirmed at visualization: - National Grand Mean: **$0.944 trillion** - Top 5 states by Grand Mean: CA $293.7B · FL $112.0B · NY $53.9B · TX $38.6B · HI $35.6B - Top 5 counties: Los Angeles $49.7B · Orange $38.5B · Santa Clara $33.8B · San Diego $30.3B · Honolulu $29.7B - OLS confirmed: = 12.225, _holes = 0.053, _urban = 4.002

7.8 Cross-Phase Data Flow Summary

```
Phase 1
  Raw GPS coordinates → Spatial join (FIPS) → Economic proxy merge (USDA/FHFA)
  → RUCC classification → Baseline_Value_Per_Acre
  Output: {Py|R|Jl}_Phase1_Baseline_Golf_Valuation.csv  (~16,292 courses)

Phase 2
  Phase 1 CSV + OSM PBF (11 GB) → Polygon extraction → Two-tier acreage match
  → acreage_source flag
  Output: {Py|R|Jl}_Phase2_Acreage_Matched.csv  (71.2% matched, 28.8% MICE_Target)

Phase 3
  Phase 2 CSV → MICE (M=100 per language) → 100 complete datasets per language
  → Rubin's Rules pooling → National aggregate ~$940B
  Output: 300 imputed CSVs + Rubins_Rules_Summary + National_Acreage_Summary

Phase 4
  300 imputed CSVs → OLS (per dataset) → HC1 robust SEs
  → Rubin's Rules pooling per language
  → ^_Python, ^_R, ^_Julia
  Output: {Py|R|Jl}_Regression_Results.csv

Phase 5
  Phase 1/2/3 outputs + Honolulu cadastral GeoPackage + Zoning GeoPackage
  → Parcel intersection → Economic validation → Geographic/zoning breakdown
  → Oahu q̄ = $25.40B; 81.7% preservation-zone; 63.2% Ewa district
  Output: Phase5 comparison, geographic, zoning CSVs

Phase 6
```

```
All upstream outputs → Grand Mean aggregation → Tri-language visualizations
→ National maps, county maps, Oahu maps, forest plots, density overlays,
   advanced structural charts, LaTeX tables
→ $0.944T confirmed national estimate
Output: ~40 PNGs + 3 .tex files in Final_Thesis_Figures/ and QA_Verification/
```

7.9 Audit and Quality Assurance Log

All six phases underwent a sequential CLAUDE.md compliance review spanning April 30 – May 9, 2026. The review covered structural conventions (four-section layout, ALL_CAPS constants, `[METHODOLOGY]` flags, memory management, file existence guards), cross-language consistency, and I/O path integrity.

Phase	Review Parts	Key Fixes
Phase 1	1A–1D	FIPS zero-padding bug; `[METHODOLOGY]` tags on CRS transforms; Julia `world-age` error resolved by self-contained master
Phase 2	2A–2D	`acreage_source` column added to Julia and Python outputs; Julia master rebuilt as self-contained
Phase 3	3A–3D	Memory management (rm/gc) added to all three language loops; stale M=30 header comments updated to M=100; `[METHODOLOGY]` on all Rubin's pooling blocks
Phase 4	4A–4D	`import gc` and `del df; gc.collect()` added to Python model-fitting loop; parameter name divergence across three languages documented (harmless by design)

Phase	Review Parts	Key Fixes
Phase 5	5A–5D	Python float-to-string Zone bug fixed; live acreage computation replaces stale hardcoded constant; memory management in Step 3 loop; `[METHODOLOGY]` tags on missing spatial reads
Phase 6	6A–6C	Phase_6.R four-section structure fixed; stale loop variables fixed (Scripts 1 and 2); `OUT_LORENZ` naming corrected to 1.234 convention; Phase_6.jl `main()` wrapper added

---

Phase 7 write-up work in progress.